Senior Design Procedure

September, 2025

0.1 Component 1 — Foundations (Repo, Cloud Accounts, IaC, CI/CD, Security, Cost)

This component lays the spine for everything else: one repo, three environments (dev/stage/prod), cloud resources created via code, continuous integration/deployment, basic security, and cost controls. The goal is that *anyone on the team can clone the repo and deploy a working sandbox in one command*, and that we never click around in consoles to create critical resources.

Outcomes (what "done" means).

- A monorepo with Frontend (React/TS/Vite), Backend API (FastAPI/TS), Contracts (Foundry), Provider Agent (Go), and Infrastructure (AWS CDK, TypeScript).
- Three AWS environments (dev, stage, prod) bootstrapped by CDK, each with: VPC, Aurora PostgreSQL Serverless v2, S3 buckets (artifacts, logs), CloudFront (artifact CDN), Cognito (auth), SQS queues (jobs/qc/settle), EventBridge schedules (timers), one small EC2 instance for the API, and Budget alarms.
- CI/CD pipelines (GitHub Actions) that lint, test, build, deploy CDK stacks, and publish web assets.
- Least-privilege IAM roles, GitHub OIDC to AWS (no long-lived keys), secrets stored in AWS Secrets Manager, KMS encryption on data-at-rest.
- Local developer workflow: run Postgres + S3 emulator locally, run API/FE locally, and switch to Aurora/S3 on stage/prod.

Prerequisites (once, centrally).

1. Create (or reuse) an AWS Organization account with three member accounts (or three profiles in one account with clear prefixes): `ngh-dev`, `ngh-stage`, `ngh-prod`.
2. Create an AWS *Billing budget* that emails the team at $50 and $80 per month (dev+stage combined), and $100 for prod (demo month only).
3. In GitHub: one private organization repository `gh:org/compute-exchange`.

Repository layout (scaffold).

```
compute-exchange/
  apps/
    web/            # React 18 + TS + Vite + shadcn/ui
    api/            # FastAPI (REST) + Prisma
    agent/          # Go 1.22 provider runtime
    contracts/      # Foundry (Solidity) for Voucher1155, Registry, Settlement
  packages/
    shared/         # Shared TS types: JobId, PackageId, VoucherKey, DTOs (zod)
    canon/          # Canonicalization lib (TS) for tabular/vector outputs
    cdk/            # AWS CDK (TS) stacks (one app, env-targeted)
  infra/
    github/         # GitHub Actions workflows
  .tool-versions    # (optional) asdf pinning Node/Go
  turbo.json        # turborepo
  pnpm-workspace.yaml
```

Step A — Initialize the monorepo.

1. Install toolchain: Node 20 (`nvm` or `asdf`), pnpm, Go 1.22, Foundry (`foundryup`), AWS CLI v2, CDK v2.
2. Create turborepo: `pnpm dlx create-turbo@latest` (empty preset). Add packages/apps as above.
3. Add linting/formatting: ESLint + Prettier (TS), golangci-lint (Go).
4. Add `packages/shared` with strongly-typed IDs (`JobId`, `PackageId`, `VoucherKey = {region,hour,sla,tier` and Zod DTOs for all v1 endpoints.

Step B — Infrastructure as Code (CDK). *We will never create resources manually. All infra is defined in `packages/cdk`.*

1. **Bootstrap CDK** in each account/profile (once): [language=bash] cd packages/cdk cdk bootstrap aws:$//<\text{ACCOUNT}_ID>/<REGION>--profilengh-devcdkbootstrapaws:$ $//<ACCOUNT_ID>/<REGION>--profilengh-stagecdkbootstrapaws:$ $//<ACCOUNT_ID>/<REGION>--profilengh-prod$
2. **Define stacks** (TS constructs) with explicit resource names and tags:
   - **NetworkStack**: 2 AZ VPC, public subnets for ALB/CloudFront origin, private subnets for Aurora and EC2 API; VPC endpoints for S3.
   - **DataStack**: Aurora PostgreSQL Serverless v2 (min 0.5 ACU), RDS Proxy (optional if using Lambda later), Secrets Manager secret for DB creds, KMS for RDS.
   - **StorageStack**: S3 buckets (`ngh-artifacts`, `ngh-logs`) with versioning, default encryption (SSE-S3 or SSE-KMS), lifecycle rules (move to IA after 7 days).

- **EdgeStack**: CloudFront distribution in front of `ngh-artifacts`, origin access control (OAC), cache policy for large binaries, HTTP range requests enabled.
- **AuthStack**: Cognito user pool + hosted UI (email login); user groups: `buyer`, `provider`, `admin`.
- **QueueStack**: SQS queues: `jobs-queue`, `qc-queue`, `settle-queue` (DLQs attached); EventBridge schedules for cron (e.g., index print, gate-close tick).
- **ComputeStack**: EC2 t3.small for API (Amazon Linux 2023), security groups, ALB; optional Lambda for small tasks (relay checksum publisher).
- **BudgetStack**: Cost & Usage budget with SNS email to team at thresholds.

3. **Environment parameters:** expose `REGION, DB_SECRET_ARN, S3_ARTIFACTS_BUCKET, CLOUDFRONT_DIST_ COGNITO_POOL_ID, QUEUE_URLS`.
4. **Deploy dev:** [language=bash] cd packages/cdk cdk deploy –all –profile ngh-dev Repeat for stage/prod when ready.

Step C — Backend API skeleton (FastAPI + Prisma).

1. **Create service:** `apps/api`. Add env config loader (dotenv for dev; SSM/Secrets Manager in cloud).
2. **Prisma schema (initial tables):** [language=sql] model User id String @id @default(cuid()) email String @unique role String createdAt DateTime @default(now()) model Job id String @id @default(cuid()) windowStart DateTime region String tier String sla String status String createdAt DateTime @default(now()) packages Package[] model Package id String @id @default(cuid()) jobId String determinism String outputSchema String requiresTier String status String firstOutputAt DateTime? gpuTimeHoursRaw Decimal? @db.Numeric(18,6) qMultiplier Decimal? @db.Numeric(9,6) paidNgh Decimal? @db.Numeric(18,6) model VoucherEscrow id String @id @default(cuid()) jobId String key String amount Decimal @db.Numeric(18,6) consumed Decimal @db.Numeric(18,6)
3. **Migrations:** local Postgres first; Aurora later. Provide `prisma migrate dev` and `prisma migrate deploy`.
4. **Healthcheck endpoint:** `GET /health` returns DB connectivity and build SHA.

Step D — Frontend shell (React/TS/Vite).

1. **Create app:** `apps/web`. Add shadcn/ui, TanStack Query, React Router.
2. **Auth integration:** Cognito hosted UI; upon login, store ID token; add role-based guards (buyer/provider/admin).
3. **Pages (stubs):** Jobs, Packages, Vouchers, Providers, Orderbook, Admin.

Step E — Contracts workspace (Foundry).

1. **Scaffold:** `forge init contracts`. Add `Voucher1155.sol`, `ReceiptRegistry.sol`, `Settlement.sol` (interfaces only for now).

2. **Deploy scripts:** `script/Deploy.s.sol` for Base/Arbitrum Sepolia. Store deployed addresses in `packages/shared`.
3. **Local tests:** minimal constructor tests and role checks.

Step F — GitHub Actions (CI/CD) and OIDC to AWS.

1. **Configure OIDC:** In AWS IAM, create an identity provider for GitHub, allow role assumption by repo `org/compute-exchange`. Attach least-privilege policies (CDK deploy, S3 upload, invalidation).
2. **Workflows:**
   - `ci.yml`: lint + test + build all packages on PR.
   - `deploy-dev.yml`: on push to `main`, CDK `deploy -all` to dev, then build & upload web to S3, then CloudFront invalidation.
   - `deploy-stage.yml`: on tag `vX.Y.Z-rc`, deploy to stage.
   - `deploy-prod.yml`: on tag `vX.Y.Z`, deploy to prod (manual approval).

Step G — Security baselines (simple, enforceable).

1. **Secrets:** DB passwords in Secrets Manager (rotation optional later). Do not store secrets in GitHub; use OIDC + parameter store for environment values.
2. **Encryption:** Enable SSE-KMS for S3 (`ngh-artifacts`, `ngh-logs`); enable KMS for Aurora; require TLS everywhere.
3. **IAM least privilege:** Separate roles: `api-ec2-role`, `cdk-deploy-role`, `github-ci-role`; scoped S3 access by bucket/prefix; SQS send/receive by queue.
4. **Audit:** Turn on CloudTrail and S3 access logs; bucket policies that block public ACLs; enable `Block Public Access`.

Step H — Cost controls (fit within $400 credits).

1. **Aurora Serverless v2** min 0.5 ACU; **stop non-prod EC2 nightly** via EventBridge + Lambda (start at 8am, stop at 8pm local).
2. **S3 lifecycle:** transition artifacts to Infrequent Access after 7 days; delete logs older than 30 days (dev).
3. **CloudFront:** set default TTL 1h for artifacts; enable compression.
4. **Budgets/Alarms:** monthly alarms at $50 and $80 (dev+stage) to team email/SNS; investigate immediately if breached.

Step I — Local developer workflow (speed + parity).

1. **Docker compose** for local Postgres & MinIO (S3 emulator). Provide `make dev` that launches DB + API + Web.

2. **Env switching:** `.env.local` for dev; stage/prod values pulled at runtime from SSM/Secrets Manager by the API; FE hits `/config.json` that API serves.
3. **Seed scripts:** Prisma seeds for users, a demo job, and fake packages.

Step J — Naming & IDs (shared across systems).

- **VoucherKey** := ⟨region, hour (ISO timestamp to hour), SLA, tier⟩ as a canonical string `us-east|2026-02-01T14:00Z|standard|T1`.
- **JobId/PackageId**: cuid2 strings; **ProviderFingerprint**: stable UUID from attestation (placeholder for now).
- All services import these types from `packages/shared` to avoid drift.

Acceptance tests (must pass before moving on).

1. **Cold-start to deploy:** On a fresh laptop: clone repo → `pnpm i` → `pnpm -w build` → `cdk deploy -all` (dev) → visit CloudFront URL and see web shell; hit `/health` and see DB ok.
2. **Permissions correctness:** Try to access S3 artifacts bucket from an unauthorized role: denied. Try to read DB creds without Secrets Manager permission: denied.
3. **Budgets:** Force a tiny spend (e.g., data transfer); verify budget alarm email arrives (can lower threshold temporarily).
4. **Local dev:** Run `make dev`; API connects to local Postgres; FE can log in via Cognito dev pool and list seeded jobs.

Interfaces produced (for downstream components).

- `ENV` contract: all apps can read `REGION, COGNITO_POOL_ID, API_BASE_URL, S3_ARTIFACTS_BUCKET, CLOUDFRONT_URL`.
- `packages/shared`: exported DTOs for `/jobs`, `/vouchers/deposit`, `/lots/{id}/prepare_ready`, etc.
- **Deployed addresses**: JSON with contract addresses for Voucher1155, ReceiptRegistry, Settlement (testnet).

Risks & mitigations (now, not later).

- **Drift from console edits:** Mitigation—*blocked*; CI enforces CDK drift checks on PR (failing job if drift detected).
- **Cost spikes:** Mitigation—nightly EC2 shutdown, Aurora min ACU, budget alarms, S3 lifecycle rules committed in CDK.
- **Secrets leakage:** Mitigation—OIDC to AWS (no long-lived keys), Secrets Manager, deny `GetParametersByPath` wildcards.
- **Env mismatch bugs:** Mitigation—`/config.json` served by API with exact values; FE never bakes secrets at build.

Research tie-ins (what this enables).

- **Event sourcing & settlement invariants (R8):** CDK defines the queues and Step Functions we will later use to guarantee "no double-consume / no orphan acceptance."
- **Index methodology (R2) & market microstructure (R4):** CloudFront + S3 give us deterministic artifacts for replay; Aurora tables will later store order/trade books for parameter estimation.
- **q-multiplier fairness (R1) & GPU-time calibration (R7):** Secrets and logging scaffolding allow safe ingestion of telemetry for later calibration without redoing infra.

Simple operator runbook (day one).

1. **Deploy dev:** `cd packages/cdk && cdk deploy -all -profile ngh-dev`.
2. **Open web:** Copy CloudFront domain from stack outputs, load in browser; log in via Cognito hosted UI.
3. **Verify DB:** `psql` into Aurora (dev) using Secrets Manager creds; run `SELECT now();`.
4. **Ship a change:** Push a trivial FE change to `main`; watch GitHub Actions deploy and CloudFront invalidate.

0.2 Component 2 — Jobs, Packages, Artifact Relay & Feasibility (Buyer MVP)

This component makes the platform *real* for a buyer: define what a **package** is, how artifacts are uploaded and verified, how a **job** is created, and how the portal computes **feasibility** (earliest start, NGH required, voucher gaps, and milestone checks). We keep the language simple while pinning precise contracts so later components (scheduler, QC, settlement, market) can rely on them.

Outcomes (what "done" means).

- A stable **PackageDescriptor v1** schema (JSON) with validation in the API *and* in the buyer CLI.
- A working **Artifact Relay**: S3 + CloudFront + signed URLs + per-chunk checksums + manifest with a Merkle root (integrity you can audit).
- Buyer can **create a job** with a package index (100–1000 packages), and the portal returns a **feasibility** report: earliest start, NGH required, voucher gap, and milestone sanity.
- Canonicalization library produces *identical hashes* for the same outputs regardless of OS/ordering (foundation for QC later).

Why this order. We must fix *what* runs (packages/artifacts) before we can schedule or verify it. Feasibility gives users immediate feedback and validates data paths end-to-end with small cost.

Data contracts (shared types). Add to `packages/shared` (TypeScript, exported to FE/BE/CLI/agent): [language=typescript] // Canonical IDs and keys export type JobId = string; // cuid2 export type

PackageId = string; // cuid2 export type ProviderFingerprint = string; // stable UUID/attestation export type VoucherKey = ‘$Region$|ISOHour|SLA|Tier‘;

// Determinism schemas export type Determinism = ’bit$_{e}xact'|'fp_{t}olerant'; exporttypeOutputSchema ='$ $embeddings@1'|'rollout@1'|'video@1'; //v1 : pickone$

// PackageDescriptor v1 (authoritative) export interface PackageDescriptor package$_{i}d : PackageId; image_{d}igest$ $string; //oci : //...@sha256 :< digest > (pinned)command : string[]; //e.g., ["python", "worker.py", "-$ $-arg", "val"]input_{m}anifest_{u}ri : string; //https : //.../manifest.jsonl(Relay)input_{m}erkle_{r}oot :$ $string; //0x < hex > overchunkhashesexpected_{i}tems : number; //e.g., 100_{0}00determinism :$ $Determinism; //'bit_{e}xact'|'fp_{t}olerant'tolerance? : rel_{t}ol : number; max_{u}lp : number; //requirediffp_{t}olerantcan$ $string; //known-answeritems(1output_{s}chema : OutputSchema; //describesrecordlayoutoutput_{u}ri_{p}refix :$ $string; //s3 : //bucket/prefix/requires : tier :' T1'|'T2'|'T3'; min_{v}ram_{g}b? : number; nvlink? : boolean; driver_{s}ta$ $tier? :' T1'|'T2'|'T3';$

Step A — PackageDescriptor v1: author, validate, freeze.

1. **Write a JSON Schema** (and Zod types) for `PackageDescriptor`. Enforce:
   - `image_digest` *must* be pinned by sha256 (prevents tag drift).
   - `determinism='fp_tolerant'` ⇒ `tolerance={rel_tol:1e-4, max_ulp:2}` in v1.
   - `requires.tier` present; **no** platform-side splitting if size is wrong.
   - `output_uri_prefix` must be an owned bucket/prefix (we will write receipts here).
2. **Freeze semantics:** once a job is created, package descriptors are *immutable*. Any change ⇒ new `package_id`.
3. **Server-side re-validation:** API re-validates descriptors (never trust client-only checks).

Step B — Canonicalization library (hashes that never lie).

1. Implement `@ngh/canon` (TS) with:
   - Deterministic sort (stable, locale-agnostic).
   - Numeric formatting with fixed precision; ULP clamping rules (for `fp_tolerant` compares).
   - Streaming Merkle builder: hash per chunk (e.g., 4–16MB), then Merkle-root over ordered chunk hashes.
   - Exact handling for NaN/Inf if schema allows; otherwise forbid.
2. Provide a `cli` that:
   - Computes `input_merkle_root` for a given manifest.
   - Verifies `output_root` for a produced shard (used later by QC).
3. **Test corpus:** same inputs across Linux/macOS produce *identical* roots; record fixture hashes.

Step C — Artifact ingestion: manifests & canaries.

1. **Input manifest format:** JSON Lines (or Parquet) with one record per item: [language=json] "key":"s3://alice/dataset/shard$_0$001.$parquet$", "$offset$" : 0, "$length$" : 1048576, "$sha256$" : "..."
2. **Canaries:** Reserve $\sim 1\%$ of items with known answers; store at `canaries_uri`; include a small `.proof.json` with expected outputs (schema-dependent).
3. **Chunking:** For large files, the manifest should reference ranges (offset/length) so providers can prefetch first chunks fast.
4. **Compute roots:** Use the canon CLI to compute `input_merkle_root` from the manifest's per-chunk checksums.

Step D — Artifact Relay (HTTP-only, checksum-first).

1. **S3 buckets:** `ngh-artifacts` (artifacts, manifests, canaries), `ngh-logs` (provider logs). Default encryption + bucket policies *deny* public ACLs.
2. **CloudFront in front of `ngh-artifacts`** with:
   - Origin Access Control (no direct S3 public access).
   - Range requests enabled, default TTL 1h, large object tuning.
3. **Relay API (FastAPI or Lambda):**
   - `GET /relay/sign?uri=...` returns short-lived signed URL for S3 object (and `HEAD` metadata).
   - `GET /relay/manifest?uri=...` streams the manifest (optionally via CloudFront).
   - `GET /relay/checksum?uri=...` returns recorded sha256 for quick integrity checks.
   - Rate-limit and require the caller to own the bucket/prefix for writes.
4. **Edge caches:** Set `Cache-Control` and `ETag` on S3 objects; record checksums in object metadata.

Step E — API endpoints (buyer-facing, v1).

1. `POST /jobs`: body includes `job_id`, window (region, ISO hour, SLA, tier), and a **package index** (list of `PackageDescriptor`).
2. `GET /jobs/{id}/feasibility`: returns:
   - **NGH required**: sum of package size estimates (see Step F).
   - **Earliest start**: next feasible window given artifact sizes and *Standard* prepare rules.
   - **Voucher gap**: NGH required minus vouchers already deposited for the key.
   - **Milestone sanity**: flags if any package likely misses *first-output* ≤ 2 *min* or is out of size band [5, 15] NGH.
3. `GET /jobs/{id}`: status summary; immutable descriptors; Relay links for manifests.

Step F — Feasibility math (simple, honest, documented). We keep the first pass transparent and conservative:

1. **Estimate package NGH**:

$$\widehat{\mathrm{NGH}}_i \;=\; \frac{\text{estimated_gpu_seconds}_i}{3600} \times \widehat{q}$$

   where $\widehat{q} = 1.0$ for v1 (we introduce per-schema q later). For estimated_gpu_seconds$_i$, use:

$$t_{\text{gpu}} \approx t_{\text{compute}}(W;\ \widehat{r}_{\text{tier}}) \quad \text{with} \quad \widehat{r}_{\text{tier}} \text{ from a published lookup table}$$

   ($\widehat{r}_{\text{tier}}$ = expected throughput for the chosen tier; e.g., tokens/s or items/s).

2. **First-output sanity:** compute

$$T_1 \approx t_{\text{pull}}(I) + t_{\text{prefetch}}(C, d) + t_{\text{warmup}} + \frac{w_0}{\widehat{r}_{\text{tier}}}$$

   and warn if $T_1 > 120$ s. Use defaults: t_{pull} from image size/bandwidth; $\{C, d\}$ (chunk size, depth) from Relay defaults; w_0 = items needed to emit first outputs.

3. **Earliest start:** round up to the next hour that allows:
   - image pull + prefetch within the ≤ 3 min prepare window before T_0,
   - total NGH within nominated capacity (if nominations are known; otherwise just display the job's own readiness).

4. **Voucher gap:**

$$\text{gap} = \sum_i \widehat{\mathrm{NGH}}_i \;-\; \text{deposited_ngh}(\texttt{VoucherKey})$$

Explain clearly in the portal: these are *estimates* for planning; the platform never edits package contents or splits them.

Step G — Database migrations (Prisma). Add/extend models: [language=sql] model Job id String @id @default(cuid()) windowStart DateTime // ISO hour (T0) region String sla String // 'standard' tier String // 'T1' status String // 'created'|'placed'|'running'... packages Package[] createdAt DateTime @default(now()) model Package id String @id @default(cuid()) jobId String @index imageDigest String command String inputManifestUri String inputMerkleRoot String expectedItems Int determinism String // 'bit$_e xact'|'fp_t olerant'toleranceRelTolFloat?toleranceMaxUlpInt?outputSchemaStringou$

Step H — Frontend flows (simple, helpful).

1. **Create Job** page: pick region/hour/SLA/tier; upload a package index (JSON or drag a folder that the CLI generated); show *live validation* (green checks) and clear errors (red).
2. **Feasibility panel:** display NGH required, earliest start, voucher gap. Include small info icons:
   - "NGH is H100-equivalent hours; we convert raw GPU time with a performance factor later."
   - "First outputs within 2 min: early sign the run is healthy."
   - "Prepare window 3 min: image pull/prefetch before T_0, billing starts at T_0."

Step I — Security & integrity (must-haves).

- **Pinned images:** reject any `image_digest` without a sha256; optionally require Sigstore/COSIGN signature (we'll enforce fully in Component **??**).
- **Bucket ownership:** only allow `output_uri_prefix` to buckets/prefixes owned by the user's account; deny writing outside.
- **Checksums everywhere:** S3 objects store `x-amz-meta-sha256`; Relay exposes `/checksum`; manifests list per-chunk sha256; `input_merkle_root` recorded in DB.
- **Immutability:** once a job is created, descriptors are frozen (status guard in DB).

Step J — Acceptance tests (demo or it didn't happen).

1. **Upload & root:** Upload a 1GB artifact; manifest lists 256 chunks; Relay returns matching checksums; `input_merkle_root` stable across two different machines.
2. **Create job:** 20 packages load, pass validation; API stores immutable descriptors; `/feasibility` returns NGH sum and no fatal flags.
3. **Milestone sanity:** Force a huge image (e.g., 10GB) in a test package; feasibility flags $T_1 > 120s$.
4. **Permissions:** Try to set `output_uri_prefix` to another user's bucket: API rejects request.

Step K — Risks & mitigations.

- **Slow first outputs due to cold caches:** Show a warning when image size $\gg$ 2GB and advise prewarming (repeat pulls, or smaller base images). Relay chunking lowers prefetch time.
- **Schema drift:** Single source of truth for types in `packages/shared`; API rejects unknown fields; version field `"descriptor_version":"1"` reserved.
- **Broken manifests:** Server recomputes random chunk hashes on upload to spot-check integrity; if mismatch, the job is quarantined.

Research tie-ins (started now, finishes later).

- **R6 Canonicalization:** the lib here becomes the exact function QC uses later; record false-reject/accept stats on a gold corpus.
- **P1 Package sizing:** feasibility collects throughput estimates that feed the packager advisor (later component) to hit 5–15 NGH with milestone success.
- **R1 q-multiplier:** capture item-throughput telemetry during dry runs to seed per-tier $\widehat{r}_{\text{tier}}$ tables.

Operator notes (day-to-day).

1. If feasibility keeps flagging $T_1 > 120s$, check the `ngh-artifacts` CloudFront cache hit-rate and reduce image sizes.

2. If users see checksum failures, verify S3 metadata and that the manifest's `sha256` lines match uploaded object hashes.
3. For support, ask users to paste the `input_merkle_root` and the first 3 chunk hashes—this diagnoses 90% of issues fast.

0.3 Component 3 — Vouchers (ERC–1155), Deposit Escrow, Day–Ahead & Gate Close

This component builds the **financial→physical bridge**. Buyers acquire *vouchers* (physically deliverable NGH for a specific hour/region/SLA/tier), *deposit* them against a job, and the platform *freezes* jobs at $T-5$ minutes (*gate close*) so they can be placed and prepared. Everything here is intentionally simple and auditable.

Outcomes (what "done" means).

- A deployed **ERC–1155 Voucher** contract with a canonical mapping from a voucher key $\langle \text{region}, \text{ISO_hour}, \text{SLA}, \text{tier} \rangle$ to a tokenId, integer *subunits* (μNGH), and basic admin mint/burn for day–ahead.
- A **deposit** workflow: buyers lock vouchers on–chain and create a mirrored *escrow ledger* row bound to a `job_id`. Idempotent; partial deposits allowed; withdrawals allowed *until* gate close.
- **Gate close** orchestration at $T-5$ minutes: freeze the job, compute the list of *lots* (one per package), and enqueue them for placement (Component 0.4).
- **Invariants recorded**: (I1) no voucher consumed twice; (I2) deposit NGH $\leq$ wallet balance locked; (I3) voucher key matches job window; (I4) after gate close, no withdraw; (I5) consumption is 1:1 with *accepted* NGH at settlement (enforced in Component 0.6).

Plain explanation (why this matters). Vouchers are like prepaid, hour–specific electricity: they *guarantee* you have capacity when the hour comes. Depositing a voucher is like putting your ticket at will–call for a specific concert time; once the doors are about to open (gate close), we stop changes and seat everyone.

Terminology & fixed units.

- **VoucherKey** $\triangleq$ canonical string

  VoucherKey := `<region>|<ISO_hour>Z|<sla>|<tier>`

  e.g., `us-east|2026-03-10T14:00Z|standard|T1`.
- **Subunits**: an ERC–1155 token represents integer subunits of NGH. We fix $\mathbf{10^6}$ subunits = **1.000000** NGH (*microNGH*) so we can deposit fractional NGH precisely.

Step A — ERC–1155 Voucher contract (Foundry).

1. **TokenId mapping.** Deterministic 256–bit id: [language=solidity] // Pseudocode: keccak256 hash of canonical key string function tokenIdOf(string memory key) public pure returns (uint256) return uint256(keccak256(bytes(key)));

2. **Subunits.** Store balances in integers of μNGH. Expose a helper: [language=solidity] uint256 public constant DECIMALS = 1e6; // microNGH function toSubunits(uint256 nghTimes1e6) internal pure returns (uint256) return nghTimes1e6;
3. **Mint/Burn hooks (day–ahead).** Minimal admin role (Ownable or AccessControl) to `mint(to, tokenId, amount, data)` and `burn(from, ...)` for physically deliverable issuance and settlement burn.
4. **Deposit lock bit (optional v1).** A simple on–chain `mapping(bytes32 jobId => uint256 locked)` is *not* required if you escrow off–chain; but add a per–wallet lock bitmap if you want belt–and–suspenders (see Step C for the off–chain ledger which is authoritative for the job).

Step B — Day–ahead minting (simple admin tool).

1. **Faucet/deployer script** (Foundry) to mint a given NGH amount (in subunits) for a voucher key to a wallet (for dev/test).
2. **Admin CLI** to batch–mint for real day–ahead: reads CSV rows $\langle \text{VoucherKey}, \text{amountNGH} \rangle$ and calls `mint` to buyers or the venue treasury (for secondary listing).
3. **Event indexing.** Record `Minted(VoucherKey, tokenId, amount, to)` in Aurora for UI and reconciliation.

Step C — Deposit API & escrow ledger (idempotent).

1. **DB model (extend Prisma).** [language=sql] model VoucherEscrow id String @id @default(cuid()) jobId String @index voucherKey String @index // region|hour|sla|tier amount Decimal @db.Numeric(18,6) // total deposited NGH consumed Decimal @db.Numeric(18,6) // consumed by settlement txHash String? // last on-chain tx involved in deposit/lock createdAt DateTime @default(now()) updatedAt DateTime @updatedAt status String // 'open'|'locked'|'closed'
2. **Endpoint:** `POST /vouchers/deposit` [language=json] "job$_id$" : "job_x", "$voucher_key$" : "$us - east|2026 - 03 - 10T14 : 00Z|standard|T1$", "$amount_ngh$" : 250.000000, "$wallet$" : "$0xBuyer$", "$nonce$" : "$uuid - ...$"
3. **Flow:**
   (a) Verify voucherKey $\equiv$ job window (region/hour/SLA/tier).
   (b) Convert NGH to subunits; compute tokenId = keccak256(voucherKey).
   (c) *Client–side* approves Voucher1155 for a small `DepositEscrow` helper contract *or* uses `safeTransferFrom` directly to a venue escrow wallet. Keep v1 simple: **venue escrow wallet** receives the tokens.
   (d) On receipt (listen to `TransferSingle` for that tokenId), the API creates/updates `VoucherEscrow` with `amount` and marks `status='open'`.
   (e) Idempotency: key on $\langle \text{jobId}, \text{voucherKey}, \text{nonce} \rangle$.
4. **Withdraw (pre–gate close only).** `POST /vouchers/withdraw`: reverse transfer from escrow wallet back to buyer; mark escrow row decreased; *blocked* if job is gate–closed.

Step D — Voucher invariants (enforced now).

- **I1 (No dup consumption):** escrow row has a `consumed` counter; Component 0.6 will increase it exactly–once per accepted package until it equals `amount`.
- **I2 (Deposit $\leq$ balance):** on deposit, check venue wallet received at least the subunits requested (by tx receipt); else reject and roll back.
- **I3 (Key matches job):** voucherKey $\stackrel{!}{=}$ job's key; otherwise 400–error. No flexible keys in v1.
- **I4 (Freeze at gate close):** escrow `status='locked'` at $T-5$ min; withdraw endpoint disabled.

Step E — Gate close orchestration ($T-5$ min).

1. **Event scheduling.** When a job is created, schedule an EventBridge rule for job.windowStart$-$5 min that triggers `/jobs/{id}/gate_close`.
2. **Endpoint:** `POST /jobs/{id}/gate_close` performs:
   (a) Validate job still in `'created'` or `'ready'`.
   (b) Set job `status='gate_closed'`; set all `VoucherEscrow.status='locked'`.
   (c) Build a **lot list**: one lot per package with ≈ 10 NGH target, containing $\langle$packageId, requires, sizeNghEstin
   (d) Enqueue lots to `jobs-queue` for placement (Component 0.4); emit an audit event.
3. **FE behavior.** Disable edits; show a banner "Gate closed: preparing providers."

Step F — APIs (complete for this component).

- `POST /vouchers/deposit`, `POST /vouchers/withdraw` (guarded by gate close).
- `GET /jobs/{id}/escrow` $\rightarrow$ list voucher rows, `amount`, `consumed`, `status`.
- `POST /jobs/{id}/gate_close` (admin/system only; invoked by scheduler timer).

Step G — Frontend (buyer portal).

1. **Escrow widget:** shows NGH required vs Deposited (from Component 0.12); button "Deposit vouchers" invokes wallet + deposit flow; progress bar for coverage (0% $\rightarrow$ 100%).
2. **Timeline chip:** "Gate closes in 00:05:00" with countdown; flips to "Gate closed" state.
3. **Receipts tab (placeholder):** explains that accepted NGH will *consume* vouchers 1:1 at completion.

Step H — Acceptance tests.

1. **Happy deposit:** Mint 250.000000 NGH on testnet; deposit 200.000000 to `job_id`; venue wallet balance rises by exactly 200,000,000 subunits; DB row created with `amount=200.000000`.
2. **Key mismatch:** Attempt to deposit `us-west|...` to a `us-east` job $\Rightarrow$ API 400 (rejected).

3. **Withdraw before gate:** Withdraw 50.000000 NGH ⇒ venue sends back subunits; row `amount` decremented; wallet matches.
4. **No withdraw after gate:** Call withdraw post–gate ⇒ 409/Forbidden; row `status='locked'`.
5. **Idempotency:** Repeat the same deposit (same nonce) ⇒ no double credit; a new nonce increments as expected.

Step I — Security & failure handling.

- **DvP later, escrow now.** In v1, the venue escrow wallet holds vouchers; later (Component 0.7) the market's DvP ledger will atomically move vouchers on trade. Keep the escrow wallet *hardware–backed* or use a custodial key service.
- **Reorg resilience.** Wait k confirmations (e.g., k=5) before marking a deposit final in DB. Store `txHash` and `blockNumber`.
- **Retry/idempotency.** All mutations carry `Idempotency-Key` header; server stores last result for that key for 24h.
- **Clock drift.** Gate–close uses server time (NTP); FE countdown is advisory only.

Step J — Research tie–ins (what this component enables).

- **R8 Atomicity.** The escrow ledger + gate–close flags are the inputs to the formal invariant "no double–consume/no orphan acceptance" we will prove in Component 0.6.
- **R3 Forwards/basis.** Day–ahead minting events and deposit coverage rates provide historical data for congestion and convenience yield estimation.
- **R2 Index eligibility.** Locked deposits per hour give an early proxy of depth for index reliability thresholds.

Operator runbook (daily).

1. **Mint day–ahead:** Run the admin CLI with the CSV; verify `Minted` events and the Aurora mirror table.
2. **Monitor deposits:** Dashboard of Deposited/Required per job; reach out to buyers under 90% coverage two hours before gate–close.
3. **Gate close audit:** EventBridge logs show `/gate_close` calls succeeded; jobs moved to `'gate_closed'`; lots enqueued.

Common pitfalls & cures.

- **Wrong key string formatting** ⇒ tokenId mismatch. *Cure:* always build key with a single helper in `packages/shared`; unit test sample keys.
- **Decimals confusion.** μNGH integer math only. *Cure:* convert in one place (server), display with fixed 6 decimals.

- **Late deposits.** Users attempting to deposit after gate–close. *Cure:* FE blocks action; message explains spillover will settle at spot index if short.

Forward links

The lots enqueued at gate close are consumed by the **Scheduler & Provider Runtime** in Component 0.4. Voucher consumption and payout occur in **Settlement & Anchoring** (Component 0.6), and trading/price discovery for vouchers lives in **Market & Spot Index** (Component 0.7).

0.4 Component 4 — Scheduler, Waves & Provider Runtime (Prepare → Execute → Reassign)

This component turns deposited vouchers and frozen jobs into real compute: we **assign packages to providers**, enforce the **prepare window** (≤ 3 minutes), launch in **waves** at T_0, watch for **first outputs** (≤ 120 seconds), and **reassign whole packages** on any miss. The provider agent is a small Go binary that pulls the image, prefetches inputs, runs the container, and reports results & telemetry. We keep the design simple, observable, and deterministic.

Plain explanation (what happens in human terms). At *gate close* (five minutes before the hour), we make a seating chart: which sealed package runs on which provider. Providers get a short "get ready" period to pull the container image and the first data chunks. When the hour starts, we press "go" on a first wave that is big enough to comfortably hit the deadline. Two minutes later, if everything looks healthy, we start a second wave. If any seat fails to produce early output (the first two minutes), we cancel that whole package and move it to a standby seat. We never split a package.

Outcomes (what "done" means).

- A working **scheduler** that, at gate close, assigns each package to an eligible provider (meets `requires.*`), creates a *lot* record, and sends a `prepare` command.
- A **provider agent** (Go) that passes preflight checks, pulls images, prefetches inputs, reports `prepare_ready` within ≤ 3 minutes, streams heartbeats, and submits `result` (including `gpu_time_hours_raw`, `output_root`).
- A **wave controller** that launches Wave–1 at T_0, Wave–2 at T_0+2 minutes (after throughput confirm), and **reassigns whole packages** on a first–output miss or agent fault.
- **Metrics & alerts**: first–output p95, reassignment rate, prepare-time SLA tracking; dashboards in CloudWatch.

Data model (Prisma additions). [language=sql] model Provider id String @id @default(cuid()) displayName String region String tier String // 'T1','T2','T3' vramGb Int nvlink Boolean driverStack String // e.g., "cuda12.4+cuDNN9" diskReadMbps Int arch String // e.g., 'sm90' heartbeatAt DateTime? status String // 'active'|'draining'|'offline' fingerprint String @unique // attestation-derived UUID (placeholder in v1)

model Lot id String @id @default(cuid()) jobId String @index packageId String @index providerId String? @index status String // 'created'|'placed'|'preparing'|'ready'|'running'|'accepted'|'rejected'|'reassigned'|'cance

sizeNghEstimate Decimal @db.Numeric(18,6) requiresTier String firstOutputDue DateTime? // T0 + 120s (when running) startedAt DateTime? finishedAt DateTime? reassignedFrom String? // previous providerId

model RuntimeReport id String @id @default(cuid()) lotId String @index providerId String @index wallTimeSec Int gpuTimeHoursRaw Decimal @db.Numeric(18,6) itemCount Int outputRoot String logsUri String preparedAt DateTime? firstOutputAt DateTime? createdAt DateTime @default(now())

Step A — Provider registry & eligibility.

1. **Register providers** (admin UI or API):
   - Region, tier, VRAM, NVLink, driver stack, disk read, arch. In v1, this is self–declared; later, we add attestation.
   - Status: `active`, `draining`, `offline`.
2. **Eligibility filter** (server function): a provider is eligible for a package if it satisfies `requires.tier` and declared capability minima (VRAM, driver, NVLink flag, disk read, arch if pinned).

Step B — Scheduler at gate close (assignment & prepare).

1. **On `/gate_close`** (from Component 0.3):
   (a) Build lots: one per package.
   (b) **Assignment (greedy v1):** [language=typescript] // Pseudocode for (const lot of lots) const candidates = eligibleProviders(lot.requires).sort(byLeastLoadedThenVRAM); const p = pickBest(candidates); // tie-break preset order assign(lot, p); // Lot.providerId = p.id, status='placed' enqueuePrepare(lot.id, p.id); // SQS -> agent
   (c) **Prepare command**: send an SQS message to the provider's queue: `PREPARE{lotId, imageDigest, command, manifestURIs, firstChunkHints}`.
2. **Standing capacity (optional v1):** mark a small fraction of providers as `standby=true`; they receive no Wave–1 lot but are ready to accept reassignments instantly.

Step C — Provider agent (Go, single binary).

1. **Preflight** (on `PREPARE`):
   - Verify GPU present; driver version matches; VRAM threshold met.
   - Disk space/bandwidth check; network egress only to allowlist (S3/Relay).
2. **Pull container image** by digest (Docker) and **prefetch** first input chunks (HTTP range requests). Target: `prepare_ready` ≤ 3 minutes from receipt.
3. **Call `/lots/{id}/prepare_ready`** with timestamps and a `ready_token` (nonce for later correlation).
4. **On `RUN`** (at T_0 wave launch): start the container with cgroup limits, mount a scratch volume; stream **heartbeats** every 15s (`stdout/bytes, items/processed`).

5. **First output detect**: watch the output prefix or stdout markers; call `/lots/{id}/first_output` immediately when the first shard is persisted.
6. **On completion**: compute `output_root` (using the shared canonicalization lib), collect counters, upload logs to S3, and POST `/lots/{id}/result`.
7. **On cancel**: stop container; clean volumes; acknowledge cancellation.

Step D — Wave controller (launch & reassignment).

1. **Wave–1 at T_0**: launch enough lots to achieve $\sim 15\%$ headroom to the deadline (simple rule: start all lots unless job is oversized vs nominations).
2. **First–output SLA**: for each running lot, set firstOutputDue $= T_0 + 120\,\text{s}$.
3. **Wave–2 at $T_0 + 2$ minutes**: if $\geq 80\%$ of Wave–1 lots reported heartbeats and the aggregate throughput $>$ expected, start remaining lots.
4. **Reassignment rule (hard)**:
   - If a lot misses firstOutputDue or agent heartbeats are stale ($>$ 30s), **cancel** the lot on its provider, **pick a standby** or next eligible provider, and **relaunch the entire package**. We *never* split contents.
   - If reassigned, record `Lot.status='reassigned'` and `Lot.reassignedFrom=providerId`.

Step E — API endpoints (provider & scheduler).

- `POST /lots/{id}/prepare_ready` $\rightarrow$ {`preparedAt, imagePullSec, prefetchSec, readyToken`}.
- `POST /lots/{id}/first_output` $\rightarrow$ {`firstOutputAt`}.
- `POST /lots/{id}/result` $\rightarrow$ {`wallTimeSec, gpuTimeHoursRaw, itemCount, outputRoot, logsUri`}.
- `POST /lots/{id}/heartbeat` $\rightarrow$ {`itemsProcessed, bytesOut, gpuUtil%`}.
- `POST /lots/{id}/cancel` (scheduler only).

Step F — Telemetry & safety (must-haves).

- **Egress allowlist**: only S3/Relay HTTPS endpoints; deny DNS to arbitrary hosts (agent-level iptables or eBPF).
- **cgroups**: limit CPU/memory/io; prevent noisy neighbor effects; enforce scratch volume size caps.
- **Signed runtime reports**: agent signs its `/result` payload with a device key (v1 can be self–signed, stored for audit).
- **CloudWatch dashboards**: first–output p50/p95, reassignment rate, prepare SLA success (%), per–provider fault rates.

Step G — SQS flows (wiring).

1. **jobs–queue** messages:

```
PREPARE{lotId, providerId, imageDigest, command, manifestURIs, hints}
RUN{lotId, providerId, T0, wave:1|2}
CANCEL{lotId, providerId, reason}
```

2. **Consumer**: provider–specific workers pull messages filtered by `providerId`; backoff on failure; DLQ if agent unreachable.
3. **Idempotency**: messages carry `msgId`; server stores last-processed `msgId` per lot to avoid duplicate actions.

Step H — Acceptance tests (demo checks).

1. **Prepare SLA**: send `PREPARE`; agent responds `prepare_ready` within $\leq$ 180s; image pulled and first chunks cached (verify by logs).
2. **First output**: on `RUN`, the container emits the first shard; agent calls `/first_output` with $T \leq 120$s.
3. **Reassignment**: simulate a hang (block network) $\Rightarrow$ scheduler cancels, chooses a standby provider, relaunches; lot completes.
4. **No split**: verify a reassigned lot re–runs the *entire* package; partial outputs from the failed run are ignored.
5. **Telemetry**: `/result` includes `gpuTimeHoursRaw`, `itemCount`, `outputRoot`; values persisted and visible in FE.

Step I — Frontend (provider & buyer views).

- **Provider console**: queue of assigned lots; `prepare_ready` timestamps; live heartbeats; buttons to drain/enable; simple fault history.
- **Buyer job view**: progress bars per package; badges for "Preparing / Running / Reassigned"; first–output timer that turns green when met.

Step J — Risks & mitigations.

- **Slow image pulls**: Mitigate with smaller base images and CloudFront; prewarm by caching popular digests; show warnings in feasibility (Component 0.12).
- **Flaky providers**: Auto–downgrade provider status after repeated misses; prefer providers with recent success; keep a standby pool.
- **Clock skew**: All timing based on server time; agent reports relative durations; controller enforces absolute SLAs.
- **Data exfiltration**: Egress allowlist + per–lot bandwidth caps; logs inspected on anomalies.

Research tie–ins (what to quantify next).

- **P3 Gate–close placement & standby sizing**: replace greedy assignment with a risk–aware optimizer; choose standby fraction σ to hit deadline targets.
- **P1 Package sizing**: feed observed T_1 and throughput into the packager advisor so buyers size to the slowest allowed tier.
- **R7 GPU–time calibration**: compare agent–reported `gpuTimeHoursRaw` to profilers; drive error $< 2\%$.

Operator runbook (day-of).

1. **One hour before**: check provider heartbeat board; drain any `offline` nodes; ensure standby pool $\geq 10\%$.
2. **At gate close**: verify lots placed; `PREPARE` sent; watch prepare SLA dashboard.
3. **At** T_0: confirm Wave–1 launch; watch first–output timer; reassign any misses; trigger Wave–2 at +2m if throughput is healthy.
4. **After completion**: confirm all lots have `/result`; hand off to QC (next component) for acceptance.

0.5 Component 5 — Quality Control & Receipts (Duplication, Canaries, Spot; Roots & Bundles)

This component decides, fairly and transparently, whether each package's output is **accepted** or **rejected**. It implements the fixed v1 policy: *5% package duplication*, *∼1% canaries per package*, and *0.5% spot re-execution of items for non-duplicated packages*, with an objective comparator (`bit_exact` or `fp_tolerant`). It then emits **receipts** and computes two Merkle roots per job—`receipt_root(job_id)` and `qc_root(job_id)`—which later drive settlement (Component 0.6).

Plain explanation (what happens). As packages run, the platform quietly (i) re-runs a small fraction on a second provider (duplication), (ii) checks a small set of "known-answer" items (canaries), and (iii) re-executes a few items from the rest on a neutral pool (spot). We compare outputs with strict, published rules. If everything lines up, the package is accepted; if not, it's rejected and immediately re-assigned by the scheduler. Every decision leaves a paper trail: a structured *receipt* and entries in a *QC log*, all hashed into roots you—or a judge—can verify later.

Outcomes (what "done" means).

- A **sampling planner** that, for each job, deterministically chooses duplicated packages (5%), canary sets (∼1%), and spot samples (0.5% items on non-dup packages) with audit-able seeds.
- A **comparator** that enforces `bit_exact` equality or `fp_tolerant` equality (rel_tol $= 10^{-4}$, 2 ULP), using the shared canonicalization library from Component 0.12.
- A **QC pipeline**: collect duplicate outputs, canary outcomes, spot outcomes; apply the acceptance rule; notify the scheduler on rejections; write **Receipt** JSON per package.

- A **root builder** that constructs `receipt_root(job_id)` and `qc_root(job_id)` deterministically from DB rows and object hashes; a **receipt bundle** (zip) with inclusion proofs for offline verification.

Data model (Prisma additions). [language=sql] model QCPlan id String @id @default(cuid()) jobId String @index dupRateBp Int // e.g., 500 => 5.00spotRateBp Int // e.g., 50 => 0.50canaryRateBp Int // 1.00seed String // server secret salt commit createdAt DateTime @default(now())

model QCSample id String @id @default(cuid()) jobId String @index packageId String @index kind String // 'dup' | 'canary' | 'spot' itemKey String? // nullable for 'dup' which is package-scoped providerId String? // for dup second-leg status String // 'planned'|'running'|'done'|'failed' result String? // 'pass'|'fail'|'within$_{t}olerance'|'out_{o}f_{t}olerance'detailsUriString?//S3pointertodiff, logs, etc.createdAtD$

model Receipt id String @id @default(cuid()) jobId String @index packageId String @unique providerId String inputRoot String outputRoot String itemCount Int wallTimeSec Int gpuTimeHoursRaw Decimal @db.Numeric(18,6) qMultiplier Decimal @db.Numeric(9,6) // v1 may fix to 1.000000 paidNgh Decimal @db.Numeric(18,6) logsUri String qcDuplicated Boolean qcDupOutcome String? // 'match'|'mismatch$_{w}ithin_{t}ol'|'mismatch_{o}ut_{o}f_{t}ol'qcCanariesPassedBooleanqcSpotPassedBooleandecis$

model Roots id String @id @default(cuid()) jobId String @unique receiptRoot String // hex merkle root over Receipts qcRoot String // hex merkle root over QC events createdAt DateTime @default(now())

Step A — QC sampling plan (deterministic & auditable).

1. **When**: Create at gate close (after Component 0.3) or immediately after Wave–1 starts.
2. **Seeds (commit–reveal in v1.1, fixed commit in v1):**
   - Derive a plan RNG seed $S = \text{HMAC_SHA256}(\text{server_salt},\ \text{job_id} \,\|\, \text{windowStart})$.
   - Record only the *hash* of server_salt in `QCPlan.seed` for audit; reveal the salt post-settlement (v1.1). For v1 demo, keep salt internal but store the derivation method.
3. **Duplicate selection (5%):** Shuffle package IDs with RNGS; take the first $\lfloor 0.05\,N \rfloor$. For `bit_exact` packages, ensure second-leg provider matches `arch` (e.g., SM90). For `fp_tolerant`, random eligible provider in same `tier`.
4. **Canary selection (∼1% per package):** For each package, sample item keys from `canaries_uri` up to rate; record each as a `QCSample(kind='canary')`.
5. **Spot selection (0.5% items):** For non-duplicated packages only, sample item keys from their *input manifest* at 0.5% rate; schedule on neutral pool (QC-controlled provider group).

Step B — Comparator (shared rules, consistent answers).

1. **Canonicalize outputs** with `@ngh/canon` (Component 0.12). Sorting and numeric formatting must be identical across environments.
2. **Bit-exact mode:** equality on bytes (or exact floats after canonicalization); any difference ⇒ mismatch.

3. **FP-tolerant mode:** for numeric fields, require both:

$$\frac{|x-y|}{\max(\epsilon, |y|)} \leq 10^{-4} \quad \text{and} \quad \mathrm{ULP}(x,y) \leq 2,$$

applied component-wise; non-numeric fields must match exactly after canonicalization. NaN/Inf are *not* allowed unless schema documents them.

4. **Diff artifacts:** When mismatch occurs, write a compact diff (top-1% largest absolute/relative deviations, first K offending records) to S3 and save its URI in `QCSample.detailsUri`.

Step C — QC pipeline (events & decisions).

1. **Duplication:** When a duplicated package finishes on its second provider, the QC service fetches both outputs, canonicalizes, compares, and sets `qcDupOutcome` in `Receipt`. Also set all corresponding `QCSample(kind='dup')` to `result='match'|...`.
2. **Canaries:** For each sampled item, re-compute output via neutral executor (or compare against expected outputs from `canaries_uri`); write `QCSample(kind='canary').result`.
3. **Spot:** For each sampled item from non-dup packages, re-exec on the neutral pool; compare; write `QCSample(kind='spot').result`.
4. **Acceptance rule (per package):**

$$\textit{canaries pass} \wedge (\text{if duplicated} \Rightarrow \text{match within tolerance}) \wedge \textit{spot checks pass} \Rightarrow \textbf{accepted};$$
$$\textit{else rejected.}$$

5. **Notify scheduler on reject:** immediately POST `/lots/{id}/cancel` with reason `'qc_fail'` so it reassigns the *whole package* (Component 0.4).

Step D — Receipts (what we record per package).

1. **On package completion**, the provider submits `/lots/{id}/result`. QC augments this with QC fields and computes `paidNgh`:

$$\texttt{paidNgh} = \texttt{gpuTimeHoursRaw} \times q, \qquad \text{v1: } q = 1.000000 \text{ (configurable later).}$$

2. **Store** `Receipt` row and also write a canonical JSON to S3 at `output_uri_prefix/receipt.json`. The canonical JSON must serialize fields in a fixed order for hashing.
3. **Audit pointers:** keep `logsUri` and, if applicable, diff links from `QCSample.detailsUri`.

Step E — Roots & receipt bundle (anchoring prep).

1. **Leaf hashing:** The leaf for each receipt is SHA256(canonical_json(receipt)). The leaf for each QC event is SHA256(canonical_json(qc_event)).
2. **Merkle construction:** Build `receipt_root(job_id)` over leaves sorted by `packageId`. Build `qc_root(job_id)` over leaves sorted by `QCSample.id`.

3. **Store roots** in `Roots` and emit an internal event `ROOTS_READY(job_id)` for settlement (Component 0.6).
4. **Receipt bundle**: write a zip (<5MB typical) to S3 containing:
   - All canonical `receipt.json` files,
   - The two root values, and
   - Inclusion proofs (Merkle branches) for every receipt and QC event.
5. **Verify script**: provide a CLI (`ngh-verify`) that: downloads the bundle, recomputes leaves, recomputes roots, and verifies inclusion proofs match the announced roots.

Step F — APIs (QC service).

- `POST /qc/plan` → create a plan row (admin/system).
- `GET /qc/plan/{jobId}` → public view of rates and anonymized selection (package IDs, item counts; seeds revealed post-settlement v1.1).
- `POST /qc/event` → append a QC event (dup/canary/spot outcome) [used internally by QC workers].
- `POST /qc/roots/{jobId}` → finalize and store `receipt_root`, `qc_root`; emits `ROOTS_READY`.
- `GET /jobs/{id}/receipts` → signed URLs for receipt bundle and per-package receipts.

Step G — Frontend (buyer & provider views).

- **Buyer**: per-package QC badges "Duplicated / Canaries / Spot "; a "Download receipt bundle" button after completion; a brief tooltip explaining tolerances for `fp_tolerant`.
- **Provider**: per-package card shows whether their output was duplicated and the comparison outcome; links to their logs.

Step H — Acceptance tests (demo-grade).

1. **Dup pass:** Run a small job with 20 packages; 1 duplicated; both legs match within tolerance ⇒ `decision='accepted'`.
2. **Dup fail:** Inject a fault in the second leg (e.g., mutate a few outputs) ⇒ comparator flags `'out_of_tolerance'`; package `'rejected'`; scheduler immediately reassigns and accepted on the new run.
3. **Canary fail:** Replace 1% known-answer outputs with wrong values ⇒ rejection + reassignment.
4. **Spot fail (non-dup):** Modify 0.5% items in a controlled run ⇒ rejection + reassignment. Overhead remains within expected bound.
5. **Roots & bundle:** Generate `receipt_root`/`qc_root`; run `ngh-verify` on another machine ⇒ roots and proofs validate.

Step I — Security, privacy & integrity.

- **Neutral pool isolation:** spot and dispute re-exec run on QC-owned providers with encrypted artifact access; same image digest and environment variables as the original run.
- **Randomness hygiene:** do not reveal plan seeds until after settlement (v1.1); log derivation method in the bundle for auditability.
- **No data exfiltration:** QC workers use the same egress allowlist as providers; all outputs go to controlled buckets.
- **Tamper-evidence:** every receipt and QC event has a canonical JSON stored; roots make retroactive edits detectable.

Research tie-ins (what to quantify next).

- **R5 Detection guarantees:** tune $(5\%, 1\%, 0.5\%)$ to detect adversaries with error rate ϵ with power $(1-\beta)$ at minimal cost; publish the chosen ϵ, β.
- **R6 Canonicalization bounds:** empirically measure false-reject/accept rates across two GPU families on a gold corpus; refine rounding/ordering rules if needed.
- **R7 GPU-time calibration:** compare `gpuTimeHoursRaw` against profilers across devices; target $|\text{error}| < 2\%$.

Risks & mitigations.

- **False rejects from float drift:** Mitigate via canonicalization + dual tolerance (relative and ULP). For `bit_exact`, strongly recommend pinning `arch`.
- **Seed leakage & gaming:** Use hidden server salt in v1; move to commit–reveal in v1.1. Keep QC pool independent from providers.
- **Throughput overhead:** Cap duplication at 5%, spot at 0.5%; throttle if capacity tight (but log the deviation in QC root).
- **Races & flapping decisions:** Accept/reject is idempotent and monotonic; once accepted, never re-open.

Operator runbook (day-of & after).

1. **Before T_0:** ensure QC plan exists; neutral pool online; canary URIs accessible.
2. **During run:** watch the "QC backlog" queue; investigate long-running duplicates; ensure spot tasks complete within budgeted time.
3. **After last package finishes:** confirm all receipts present; press "Compute roots" (or let the cron do it); download bundle and run `ngh-verify` as a sanity check.
4. **Hand-off:** after roots are recorded, settlement (Component 0.6) can consume vouchers 1:1 for *accepted* paid NGH.

0.6 Component 6 — Settlement & On–Chain Anchoring (Receipts → Vouchers → Payouts)

This component closes the loop: when QC (Component 0.5) finishes, we **anchor** the job's evidence on–chain and **settle** money and vouchers. Each *accepted* package consumes vouchers *1:1* (same region/hour/SLA/tier) for its `paidNgh`, providers are paid, and any rejections trigger slashing that funds re–procurement. The design is *package–scoped*, idempotent, and auditable.

Plain explanation (what happens). Think of two ledgers that must agree: (i) the *work ledger* (which packages finished and how much NGH was accepted), and (ii) the *voucher ledger* (what prepaid capacity was deposited). Settlement ties them: for every package we accepted, we "stamp" the on–chain log with cryptographic roots and then reduce (burn/consume) the matching vouchers and pay the provider. If a package was rejected, we don't pay and we apply a penalty that finances backup capacity.

Outcomes (what "done" means).

- Minimal contracts deployed (L2 testnet in v1): **ReceiptRegistry** (stores `receipt_root`, `qc_root`) and **Settlement** (records per–package settlement, consumes vouchers from venue escrow, emits payout/slash events). We reuse **Voucher1155** from Component 0.3.
- A **settlement service** that: (1) verifies the job has final roots; (2) iterates accepted receipts; (3) *atomically* (exactly-once) increments `consumed` in the escrow row, calls Settlement on–chain for the microNGH amount, and marks the package `settled`.
- A **provider payouts ledger** that credits providers (stablecoin on L2 in demo, or an internal balance) for `paidNgh` and debits penalties on rejections.
- **Invariants enforced**: (I1) every accepted package consumes vouchers exactly once; (I2) total consumed NGH $\leq$ total deposited NGH for the job's key; (I3) no payout without acceptance; (I4) anchoring (roots) happens before first payout.

Units & rounding (fixed). All voucher math uses micro–NGH integers:

$$\text{microNGH} := \left\lfloor \texttt{paidNgh} \times 10^6 \right\rfloor, \quad \text{with display rounded to 6 decimals.}$$

If fractional dust accumulates across packages, it remains in escrow until explicitly refunded post–job.

Data model (Prisma additions). [language=sql] model PackageSettle id String @id @default(cuid()) packageId String @unique jobId String @index providerId String @index paidNgh Decimal @db.Numeric(18,6) microNgh BigInt // integer subunits consumed voucherKey String txHash String? // on-chain settlement tx (if any) status String // 'pending'|'onchain$_{p}osted'|'final'createdAtDateTime@def$

model ProviderBalance id String @id @default(cuid()) providerId String @unique currency String // 'USDC' (demo) or 'CREDITS' available Decimal @db.Numeric(18,6) pending Decimal @db.Numeric(18,6) updatedAt DateTime @updatedAt

model ProviderTxn id String @id @default(cuid()) providerId String @index kind String // 'payout'|'slash' amount Decimal @db.Numeric(18,6) refPackageId String note String? createdAt DateTime @default(now())

Contracts (Foundry, minimal).

1. **ReceiptRegistry.sol** (anchor, once per job). [language=solidity] contract ReceiptRegistry is Ownable event Anchored(bytes32 jobId, bytes32 receiptRoot, bytes32 qcRoot); mapping(bytes32 => bool) public anchored; function anchor(bytes32 jobId, bytes32 receiptRoot, bytes32 qcRoot) external onlyOwner require(!anchored[jobId], "already"); // Optionally hash context (chainId, version) to avoid replay anchored[jobId] = true; emit Anchored(jobId, receiptRoot, qcRoot);

2. **Settlement.sol** (package–scoped, idempotent). [language=solidity] interface IVoucher1155 function burn(address from, uint256 id, uint256 amt) external;

   contract Settlement is Ownable event Settled(bytes32 pkgId, bytes32 jobId, uint256 tokenId, uint256 microNgh, address provider, bool paid); mapping(bytes32 => bool) public settled; // pkgId -> done IVoucher1155 public voucher; address public venueEscrow; // holds deposited vouchers

   constructor(address voucher1155, address escrow) voucher = IVoucher1155(voucher1155); venueEscrow = escrow;

   function settle(bytes32 pkgId, bytes32 jobId, uint256 tokenId, uint256 microNgh, address provider, bool isAccepted) external onlyOwner require(!settled[pkgId], "idempotent"); settled[pkgId] = true; if (isAccepted microNgh > 0) voucher.burn(venueEscrow, tokenId, microNgh); // consume vouchers 1:1 // Payout signal happens off-chain in v1 (emit event) emit Settled(pkgId, jobId, tokenId, microNgh, provider, isAccepted);

3. *Why events only?* In the demo, we *emit* the payout intent and finalize cash on the venue ledger (Aurora). For production, add an ERC20 transfer from a funded treasury with a daily cap & multisig.

Step A — Pre–settlement checks.

1. Confirm `Roots(jobId)` exists (Component 0.5) and that every `Receipt.decision='accepted'|'rejected'` is final.

2. Compute the voucher tokenId = keccak256(voucherKey) for the job's key (region, hour, SLA, tier).

3. Compute the *required microNGH consumption*:

$$M := \sum_{\text{accepted pkgs } i} \left\lfloor \texttt{paidNgh}_i \times 10^6 \right\rfloor .$$

   Check that escrowed balance (venue wallet + DB) $\geq M$; if not, mark a *shortfall* to be settled at the Spot Index later (Component 0.7).

Step B — Settlement flow (exactly–once). We use an idempotent, package–scoped *saga* to handle retries without double–consumption:

1. For each accepted `Receipt` not yet settled, create a `PackageSettle` row with status `'pending'` and its `microNgh`.

2. In a DB transaction:
   (a) Increment the job's `VoucherEscrow.consumed` by `microNgh`. If this would exceed `amount`, abort (safety).
   (b) Mark `PackageSettle.status='onchain_posting'`.
3. Call `Settlement.settle(pkgId, jobId, tokenId, microNgh, provider, true)` from the venue signer; store `txHash`.
4. When the tx reaches k confirmations, set `PackageSettle.status='final'` and credit the provider:

   ProviderBalance.available += `paidNgh`×pricePerNGH (demo: fixed price or = 1 credit per NGH).

   Emit a `ProviderTxn(kind='payout')`.
5. For rejected receipts: call `Settlement.settle(..., isAccepted=false)` (no burn), and emit a `ProviderTxn(kind='slash')` with the *imbalance fee* (policy: fixed $ per NGH or a % of the hour's index).

Idempotency: the contract guards by `settled[pkgId]`; the DB guards by the `PackageSettle.unique(packageId)` row.

Step C — Anchoring (roots first, always).

1. The settlement service *refuses* to run until `ReceiptRegistry.anchor(jobId, receiptRoot, qcRoot)` has succeeded.
2. The anchoring call is one per job; it references only hash roots, not raw receipts, keeping gas low and privacy intact.

Step D — Shortfalls, over–deposits, and dust.

- **Shortfall:** if $\sum$ microNgh > `VoucherEscrow.amount` (should be rare), mark the delta to settle at the *hourly Spot Index* (Component 0.7); debit buyer's account or invoice in demo.
- **Over–deposit:** if the job finishes with unused vouchers, keep them in escrow until the buyer clicks "*release*." API transfers remaining subunits back from venue escrow wallet to buyer.
- **Dust:** leftover subunit dust $< 10^{-6}$ NGH remains in escrow; batch refund when it aggregates > 0.01 NGH.

APIs (settlement service).

- `POST /settlement/anchor` → owner–only; forwards to `ReceiptRegistry.anchor`; records `txHash`.
- `POST /settlement/run/{jobId}` → iterates receipts and runs the saga (admin/system).
- `GET /jobs/{id}/settlement` → per–package settlement status, voucher consumed, provider payouts.

- `POST /escrow/{jobId}/release` → transfer unconsumed vouchers back to buyer (pre–defined wallet).

Frontend (what users see).

- **Buyer**: a "*Settlement*" tab showing: total accepted NGH, vouchers consumed, any spot shortfall, button to *release* remainder, and a link to the on–chain `Anchored(jobId,...)` event.
- **Provider**: a "*Earnings*" view: list of packages with `paidNgh`, payout status, slashes (if any), and downloadable *receipt bundle* for audits.

Acceptance tests (demo–grade).

1. **Anchor then settle:** Attempt settlement before anchoring ⇒ blocked. Anchor roots ⇒ settlement proceeds.
2. **Exact consumption:** 3 accepted packages with $\{10.088, 9.990, 10.001\}$ NGH ⇒ contract burns 30,079,000 microNGH from escrow; DB `consumed` increases by the same integer.
3. **Idempotency:** Re–run settlement job ⇒ on–chain guard prevents a second burn; DB rows unchanged.
4. **Reject & slash:** A rejected package triggers a `slash` entry; no voucher burn; provider balance decreases by the policy amount.
5. **Over–deposit refund:** With unconsumed escrow, `/escrow/release` transfers vouchers back to buyer; Aurora & chain events line up.

Security & failure handling.

- **Two–phase safety:** DB first (reserve consumption), then on–chain settle; on on–chain *failure*, roll back the DB transaction; on *success*, mark final. Use queue retries with exponential backoff.
- **Confirmations:** require k L2 confirmations (e.g., k=5) before crediting provider balances.
- **Key management:** venue signer in an HSM/custody service; daily spend limits; `owner` roles behind multisig in production.
- **Audit trail:** every state change emits an Aurora event and is mirrored in an append–only "`settlement_log`" table for replay.

Research tie–ins (what we quantify/prove).

- **R8 Atomicity & invariants:** model–check the saga to prove "no double–consume / no orphan acceptance"; simulate message loss/retry/reorg and show invariants hold.
- **R2 Index integrity:** the *accepted–only* consumption stream feeds the hourly Spot Index; we'll analyze manipulation bounds in Component 0.7.

- **R4 SLA option pricing:** link observed rejects & reassignments to the priced reassignment option in the Standard SLA premium.

Operator runbook (day-of and after).

1. **When QC finishes:** click "Anchor roots" (or wait for cron). Verify the chain event `Anchored(jobId,...)`.
2. **Run settlement:** start `/settlement/run/{jobId}`; watch per–package progress; confirm burns on the block explorer.
3. **Payout reconcile:** verify provider balances sum to total accepted `paidNgh` times demo price; spot–settle any voucher shortfall.
4. **Refund remainder:** release unused vouchers to buyers and close out the escrow row.

Common pitfalls & cures.

- **Rounding surprises:** always compute microNGH = $\lfloor \texttt{paidNgh} \cdot 10^6 \rfloor$ once, store, and reuse; never recompute from decimals.
- **Out–of–order calls:** block settlement until roots are anchored; require job state `'qc_complete'`.
- **Partial failures:** if the on–chain tx reverts after DB consumed increased, the saga *rolls back* the DB transaction (atomic). If the tx succeeds but DB commit fails, retry will see `settled[pkgId]` and reconcile the DB row to `'final'`.

0.7 Component 7 — Market & Spot Index (Voucher Trading, DvP Custody, Manipulation-Resistant Prints)

This component gives buyers and providers a simple, safe place to **trade vouchers** (the physically deliverable unit of compute) and publishes a **Spot Index** per (region, hour, SLA, tier). It also wires basic custody so trades are *delivery-versus-payment* (DvP) within our venue. We keep the mechanics as small as possible for the demo, but the contracts, database, and APIs are designed so you can scale out later.

Plain explanation (what this does). Think of vouchers as prepaid, hour-specific tickets for compute. The market lets anyone who holds a ticket sell it, and anyone who needs one buy it. We keep balances inside the venue (like a brokerage sub-ledger) so that when an order matches, both sides settle instantly—no one ships an asset and then waits, and no one pays and then waits. Every hour we also publish a simple "spot" price (an average of real transaction prices with guardrails) so people who didn't pre-buy know what they'll pay if they end up short.

Outcomes (what "done" means).

- A working **Voucher Market** (single matching engine) that supports *limit orders* for a given `VoucherKey` and executes with price-time priority.
- A minimal **DvP sub-ledger**: deposits/withdrawals to/from the on-chain Voucher1155 escrow (Component 0.3); instant internal moves on trade; balances per account and key.

- A published **Spot Index** per (region, hour, SLA, tier) with clear rules: VWAP, minimum depth, and outlier rejection; a *preliminary* print (real-time) and a *final* print (after hour end + acceptance).
- **Maker/taker fees** and basic surveillance hooks to deter wash trading, spoofing, and quote stuffing.

Scope & sequencing. This component can be developed in parallel with Components 0.4–0.6, but the *final* Spot Index needs acceptance signals from QC/Settlement to print its "accepted-only" VWAP. For the demo, we ship both: a *live provisional* index (from trades) and a *final accepted* index (recomputed after settlement).

Data model (Aurora; new tables). [language=sql] – Accounts custody model Account id String @id @default(cuid()) display String kycLevel String // 'none'|'basic' (demo: 'none') createdAt DateTime @default(now()) model CustodyBalance id String @id @default(cuid()) accountId String @index voucherKey String @index – region|ISO$_h our|sla|tiermicroNghBigInt --integersubunitsupdatedAtDateTime@updatedAt@@unique([accountId, voucherKey])modelCashBalanceidStr$

– Orders, trades, positions model Order id String @id @default(cuid()) accountId String @index voucherKey String @index side String // 'buy' | 'sell' price Decimal @db.Numeric(18,6) – price per NGH (currency/NGH) qtyMicroNgh BigInt – remaining quantity in subunits origMicroNgh BigInt tif String // 'DAY' (demo) status String // 'open'|'partially$_f illed'|'filled'|'canceled'createdAtDateTime@default(nou$

– Index prints (provisional final) model SpotIndex id String @id @default(cuid()) voucherKey String @unique status String // 'provisional'|'final' vwap Decimal @db.Numeric(18,6) depthMicroNgh BigInt tradesUsed Int printedAt DateTime @default(now())

– Controls surveillance model FeeSchedule id String @id @default(cuid()) makerBps Int // e.g., 0 bps takerBps Int // e.g., 8 bps minNotional Decimal @db.Numeric(18,6) // minimum notional per order tickSize Decimal @db.Numeric(18,6) lotSizeMicroNgh BigInt – e.g., 10*1e6 (10 NGH) createdAt DateTime @default(now()) model SurveillanceEvent id String @id @default(cuid()) kind String // 'wash$_s uspect'|'spoofing'accountIdString?voucherKeyStringdetailsStringcreatedAtDateTime@default(now())$

Step A — Accounts, custody, DvP sub-ledger (simple and safe).

1. **Accounts:** signup creates an `Account` row and two balances:
   - `CashBalance(currency='CREDITS')` with demo funds (or connect to testnet USDC later).
   - `CustodyBalance(voucherKey=...)` rows are created lazily on first deposit.
2. **Voucher deposit:** reuse Component 0.3 flow, but allow deposit to a *trading sub-account* instead of a job:
   (a) User transfers Voucher1155 tokens to venue escrow wallet.
   (b) API credits `CustodyBalance.microNgh` for that `voucherKey` on k confirmations.
3. **Voucher withdrawal:** only allowed if no open orders on that key and no unsettled trades; API debits the sub-ledger and transfers tokens back on-chain.

4. **DvP on trade:** when a trade matches:
   - *Buyer:* decrease `CashBalance.amount` by (price × qty) and increase `CustodyBalance.microNgh`.
   - *Seller:* decrease `CustodyBalance.microNgh` and increase `CashBalance.amount`.

   The venue escrow wallet is unchanged until users withdraw; inside the venue we maintain exact sub-ledgering for audit.

Step B — Matching engine (single book, price–time priority).

1. **Order entry rules (keep it narrow):**
   - Order types: **limit** only; `tif='DAY'`; cancels allowed.
   - Price grid: `tickSize` from `FeeSchedule` (e.g., $0.01 / NGH).
   - Quantity grid: multiples of `lotSizeMicroNgh` (default 10 NGH) for simplicity.
   - Minimum notional: `minNotional` to avoid dust orders.
2. **Risk checks before accept:**
   - *Buy:* ensure cash $\geq$ price $\times$ qty $\times(1 + \text{takerFee})$.
   - *Sell:* ensure custody $\geq$ qty.
3. **Matching:** standard price–time priority against the opposite book; partial fills create a residual order. Trades are recorded immediately.
4. **Fees:** taker pays `takerBps` (deduct from cash on buy or from cash proceeds on sell); maker gets rebate (in demo may be 0).
5. **APIs/WebSocket:**
   - REST: `POST /orders`, `DELETE /orders/{id}`, `GET /book/{voucherKey}`, `GET /trades/{voucherKey}`
   - WS: book snapshots + incremental depth updates; trade prints; index stream.

Step C — Spot Index methodology (provisional vs final). We publish two versions for each `voucherKey` (i.e., each hour and tier in a region):

- **Provisional** (*live*): VWAP over eligible trades in the last W minutes (e.g., W=30), subject to minimum depth and outlier filtering.
- **Final** (*after the hour*): VWAP using only **accepted** NGH that settled without vouchers (i.e., true spot consumption for that hour), plus eligible voucher trades executed during the hour that meet depth anti-wash rules. This aligns the index with actual delivered compute.

Eligibility & filters (both prints):

1. **Depth floor:** include trades until cumulative $\sum \text{qtyMicroNgh} \geq L_{\min}$ (e.g., $L_{\min} = 50$ NGH). If not reached, no print (index absent) or carry last print with a "no-print" flag.
2. **Outliers:** compute a robust center (median price) and MAD; drop trades with $|p - \text{median}| > 5\,\text{MAD}$.

3. **Anti-wash rules:** exclude trades where buyer and seller are the same account; flag sequences with rapid self-crossing; mark such `Trade.isIndexEligible=false`.
4. **VWAP:**
$$\text{VWAP} = \frac{\sum_i p_i \cdot q_i}{\sum_i q_i}, \quad q_i = \text{qtyNGH},\ p_i = \text{price/NGH}.$$

Final print reconciliation (accepted-only).

1. When Component 0.6 settles the hour, compute the total *accepted spot NGH* for that key (jobs that consumed spot due to voucher shortfall).
2. If this accepted spot volume is $\geq L_{\min}$, compute VWAP over those settlement unit prices; if less, blend with eligible in-hour voucher trades to reach $L_{\min}$.
3. Anchor the final index value alongside the job roots (optional, v1.1) or store in Aurora with a signed record.

Step D — Frontend (Trader view, simple and clear).

- **Market board:** cards for each hour (next 24h) by region/tier/SLA showing provisional index, book top-of-book (best bid/ask), and depth.
- **Order ticket:** choose `voucherKey`, side, price, quantity; show cash/custody checks; fees displayed up front.
- **Positions:** balances by `voucherKey` with P&L vs last index; button to *Deposit / Withdraw*.
- **Index banner:** "Provisional index $X.XX (depth: Y NGH); final prints at hh:mm after settlement."

Step E — Acceptance tests (demo-grade).

1. **DvP match:** place a resting sell, hit it with a buy; verify buyer cash decreases by notional+fees and voucher balance increases by qty; seller voucher balance decreases and cash increases by notional-fees.
2. **Risk checks:** try a buy without cash; API rejects with clear error; try a sell without vouchers; rejected.
3. **Index provisional:** generate trades with small noise around $2.00; outlier at $10.00 is filtered; provisional VWAP near $2.00 with depth $\geq L_{\min}$.
4. **Index final:** settle an hour with 80 NGH of accepted spot at $2.10 average and 60 NGH of in-hour voucher trades at $2.00; final print follows the method (prefer accepted spot, blend to depth).
5. **Anti-wash:** send self-trades; flagged and excluded from index eligibility; surveillance row recorded.

Step F — APIs (REST + WS).

- `POST /market/deposit` → credit custody after on-chain transfer confirmed (returns new `CustodyBalance`).
- `POST /market/withdraw` → debit custody and initiate on-chain withdrawal (guard: no open orders).
- `POST /orders` → place a limit order (validates risk).
- `DELETE /orders/{id}` → cancel.
- `GET /book/{voucherKey}` → aggregated depth (price levels, quantities).
- `GET /trades/{voucherKey}` → last N trades (price, qty, eligibility).
- `GET /index/{voucherKey}` → current provisional print (status, depth, methodology version); `/final` for final print after settlement.
- `WS /stream` → live order book deltas, trade prints, index updates.

Step G — Fees, lot sizes, and basic market controls.

- **Tick & lot:** set `tickSize` and `lotSizeMicroNgh` (default 10 NGH) to align with physical lots and keep the book tidy.
- **Fees:** start with maker = 0 bps, taker = 8 bps; adjust based on observed spreads; publish changes weekly (demo policy).
- **Min notional:** avoid tiny orders (e.g., $10 min) to reduce noise.
- **Kill switch:** per-key circuit breaker if provisional index deviates $> \gamma$ (e.g., 25%) from last hour without depth; pause prints and warn.

Step H — Security, anti-manipulation, and logging.

- **AuthZ:** each order requires an authenticated account; IP rate limits per account to deter stuffing.
- **Wash detection:** same account on both sides ⇒ reject; linked accounts (heuristics: shared device/keys) ⇒ flag `SurveillanceEvent`.
- **Spoofing detector:** cancel-add-cancel patterns near top-of-book logged; thresholds alert ops.
- **Audit:** immutable append-only `market`$_{l}og(KafkaorAuroraıeventsJtable)forallorder/trade/indexactions;u$

Research tie-ins (what we quantify or prove).

- **Pricing novelties:** use this market's trades to estimate congestion premium and convenience yield; publish a forward curve (v2).
- **Index integrity:** simulate attackers; tune $L_{\min}$, MAD threshold, and anti-wash rules so moving VWAP by γ costs $\gg$ expected benefit.
- **Maker incentives:** calibrate fee/rebate to hit target spread s and depth D under observed volatility.

Operator runbook (daily).

1. **Before market open:** verify fee schedule and lot/tick settings; ensure escrow listener and deposit crediting are healthy.
2. **During trading:** watch spread and depth dashboard; scan surveillance feed; respond to circuit breaker events.
3. **On the hour:** snapshot provisional index; after settlement, compute the *final* accepted-only print and store a signed record.
4. **End of day:** reconcile custody balances to escrow wallet on-chain totals (plus pending withdrawals); archive order/trade logs.

Common risks & mitigations.

- **Illiquid hours:** If depth $< L_{\min}$, don't print; show "no index—trade volume too low." Consider RFQ fallback (v1.1).
- **Index whipsaw:** Use MAD filter and depth floor; display confidence badge (green/yellow).
- **Custody drift:** Reconcile venue sub-ledger to on-chain escrow balances hourly; lock deposits/withdrawals if mismatch exceeds a tiny epsilon.

0.8 Component 8 — Provider Provisioning, Capability Registry & Nominations (Joining the Supply Side)

This component onboards **providers** (anyone offering GPUs) into the platform, records what their machines can actually do (**capabilities**), and lets them declare how much compute they will deliver in each hour (**nominations**). It gives the scheduler (Component 0.4) a clean, trustworthy list of *eligible* machines at gate close and creates the operational spine for growth: more providers $\Rightarrow$ deeper markets.

Plain explanation (what happens). If you want to sell compute, you create a provider account, install a tiny agent, and register your machine's specs—VRAM, driver, disk throughput, and so on. You then "nominate" how many normalized GPU-hours you will deliver in a specific window (e.g., us–east, Tier 1, Tue 14:00–15:00). Five minutes before the hour, the marketplace freezes the plan and assigns packages only to providers that meet the *declared* requirements. Health checks and heartbeats keep unreliable machines out of the plan. In later versions we'll add cryptographic attestation; for the demo we start with signed metadata plus strong operational checks.

Outcomes (what "done" means).

- A **provider onboarding flow** with identity, keys, and a reproducible `provider_fingerprint`.
- A **capability registry**: per-device facts (VRAM, arch, driver stack, disk/network metrics), last-verified timestamps, and simple self-tests.
- A **nomination API** for providers to declare NGH supply per (region, hour, SLA, tier); nominations are visible to the scheduler at gate close.

- **Health gating**: only `status='active'` & `last_heartbeat` fresh providers can be assigned; draining/offline are ignored.

Data model (Aurora; Prisma additions). [language=sql] model ProviderAccount id String @id @default(cuid()) displayName String email String @unique pubkeyPem String // used by agent to sign reports (v1) createdAt DateTime @default(now()) status String // 'active'|'suspended'

model ProviderNode id String @id @default(cuid()) accountId String @index region String // 'us-east', ... tier String // 'T1'|'T2'|'T3' arch String // 'sm90','sm80', ... vramGb Int nvlink Boolean driverStack String // 'cuda12.4+cuDNN9' diskReadMbps Int netEgressMbps Int fingerprint String @unique // derived at first registration status String // 'active'|'draining'|'offline' heartbeatAt DateTime? lastVerifiedAt DateTime? notes String?

model Nomination id String @id @default(cuid()) accountId String @index region String @index isoHour DateTime @index // start of the hour (UTC) sla String // 'standard' tier String // 'T1'|'T2'|'T3' qtyMicroNgh BigInt // total NGH offered across nodes (in subunits) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt status String // 'open'|'frozen'|'withdrawn'

model CapabilityCheck id String @id @default(cuid()) nodeId String @index kind String // 'disk$_read', ' net_egress', ' driver', ' vram' valueString//measuredvalue(e.g., ' 580MB/s')passedBooleandetailsUriSt$

Step A — Provider identity & fingerprint (v1).

1. **Account creation:** email + password (or OAuth) creates `ProviderAccount`; upload an **Ed25519** public key (`pubkeyPem`). The private key is used by the agent to sign heartbeats/reports.
2. **Node fingerprint (deterministic):** when the agent registers a node, it collects stable hardware facts and computes: [language=txt] fingerprint = SHA256(concat(machine$_uuid, gpu_pci_ids_orted, cpu_id, m$
   Store `ProviderNode.fingerprint`. If a provider reinstalls, the fingerprint should remain the same. (v1.1: replace with TPM/TEE quote.)

Step B — Agent install & self-test (5 minutes).

1. **Install:** `curl | bash` script downloads the agent binary; provider pastes a one-time token; agent registers and receives an API token scoped to the node.
2. **Self-test:**
   - *Driver & GPU*: query CUDA/OpenCL; verify compute capability matches `tier`.
   - *VRAM*: check free/total VRAM $\geq$ declared `vramGb`.
   - *Disk & network*: run a quick read test against a relay URL (range GETs) and a local temp file; record `diskReadMbps` and `netEgressMbps`.
3. **Post results:** `POST /providers/nodes/{id}/capability` with signed payload; backend creates `CapabilityCheck` rows and updates `ProviderNode.lastVerifiedAt`.
4. **Heartbeat:** every 30s the agent sends `/providers/nodes/{id}/heartbeat` signed with the node's key, updating `heartbeatAt`. If stale > 90s, node becomes `offline`.

Step C — Declaring capabilities (what we store).

- **Tier:** the provider chooses `'T1'|'T2'|'T3'` consistent with device class. (Platform validates: e.g., T1 ⇒ ≥80GB HBM-class, etc.)
- **Arch & driver:** used by the scheduler to satisfy `requires.arch` and determinism pins.
- **Disk & network:** used to check milestone feasibility (first-output in ≤120s).
- **NVLink flag:** relevant for single-node, multi-GPU jobs (future).

Step D — Nominations (how providers offer supply).

1. **Window picker:** provider selects (region, ISO-hour, SLA, tier). The UI shows current demand and last known spot price for context.
2. **Quantity entry:** enter a number of NGH; the API stores `qtyMicroNgh` (micro-units). The platform does *not* allocate per-node yet; just total per account per key.
3. **Freeze policy:** at *gate close* for that key, nominations become `'frozen'` and visible to the scheduler. Before gate close, providers may `withdraw` or edit.
4. **No-show handling (v1 demo):** if a provider's nodes fail prepare or miss first-output repeatedly, the account is flagged and temporarily prevented from nominating until cleared (slash accounting is handled in Component 0.6).

Step E — Eligibility & health gating (hard rules).

1. **Eligibility filter** for assignment uses:

   `ProviderNode.status='active'` ∧ heartbeat fresh ∧ capabilities ≥ package `requires.*`

2. **Draining:** when a provider flips a node to `draining`, it will finish current lots but receive no new ones.
3. **Offline:** stale heartbeat or failed self-test marks node `offline`; scheduler cannot assign it.

APIs (REST; signed where noted).

- `POST /providers/accounts` → create account; upload `pubkeyPem`.
- `POST /providers/nodes` → register node, returns `nodeId`, one-time token.
- `POST /providers/nodes/{id}/capability` [*signed*] → capability facts + self-test results.
- `POST /providers/nodes/{id}/heartbeat` [*signed*] → updates `heartbeatAt`.
- `PATCH /providers/nodes/{id}` → set `status='draining'|'active'`.
- `POST /nominations` → create or edit nomination for a (region, hour, SLA, tier).
- `DELETE /nominations/{id}` → withdraw before gate close.
- `GET /nominations?key=...` → view book depth (for transparency).

Frontend (provider console).

- **Nodes tab:** shows each node's region, tier, arch, VRAM, driver, disk/network stats, last heartbeat, and status pill (active/draining/offline). Button: "Run self-test".
- **Nominations tab:** calendar-like grid of upcoming hours per region/tier with quick-entry of NGH and color-coded demand.
- **Health banner:** warnings if self-test stale (>7 days) or heartbeats flaky; link to docs on milestones and prepare window.

Acceptance tests (demo-grade).

1. **Onboard & self-test:** register account, node, run self-test; see capability rows created; node `status='active'`, heartbeat fresh.
2. **Nominate & freeze:** nominate 100 NGH for a future hour; verify `status='open'`. At gate close, `status='frozen'`; scheduler consumes this supply.
3. **Draining respected:** set node to `draining`; scheduler assigns no new lots at the next wave; existing lot completes.
4. **Offline gating:** stop heartbeats; node flips `offline`; assignment attempts skip it.

Security & integrity (v1).

- **Signed agent calls:** all node-originated endpoints include a nonce and an Ed25519 signature over the body; the server verifies against `pubkeyPem`. Reject replays.
- **Sanity fences:** disk/network self-tests must exceed minimal thresholds (e.g., $\geq$300 MB/s disk read, $\geq$200 Mbps egress) for T1; else node cannot be `active`.
- **Abuse controls:** if a provider repeatedly fails prepare or first-output, auto-quarantine the account (status `suspended`) pending manual review.

Research tie-ins (next-layer rigor).

- **D2 Sybil resistance:** replace v1 fingerprint with TPM/TEE-backed attestation; bind device identity to stable hardware certs; quantify identity-collision risk.
- **P3 Risk-aware placement:** feed real disk/network distributions into the gate-close optimizer to predict first-output success per node.
- **P4 Normalization & q-mult:** attach a microbenchmark suite per output schema (tokens/s, images/s) and compute q; publish dispersion per tier.

Operator runbook (daily).

1. **Monitor health:** dashboard of heartbeat freshness and self-test age; auto-email providers with stale checks.
2. **Review nominations:** compare offered NGH vs expected demand by hour; recruit additional providers for thin keys.

3. **Gate-close preflight:** 10 minutes before each hour, ensure enough `active` nodes with healthy metrics; flip flaky nodes to `offline`.

Common pitfalls & cures.

- **Inflated disk/net claims:** we ignore claimed values; only *measured* self-tests count. If Relay is far, require two endpoints and take the min.
- **Key mismatch on signatures:** show a copy-paste helper to install the correct public key; provide a "rotate key" button that keeps the node identity but updates `pubkeyPem`.
- **Time-zone mistakes on hour selection:** the UI displays both local time and `ISO_hour (UTC)` to avoid misnomination.

0.9 Component 9 — Dispute Workflow & Evidence Packaging (Neutral Re→Decide →Slash/Reassign)

This component resolves **contention** when QC detects trouble (dup mismatch, canary fail, or spot fail). It runs a **neutral re-execution** of a fixed fraction of items, makes a **clear, auditable decision** (*accept* vs *reject & slash at-fault provider*), and packages **evidence** for external verification. It is time-bounded so Alice's deadline is preserved.

Plain explanation (what happens). If the quick checks say "these results look wrong," we do not argue with logs—we *re-run* a chunk of the work on a neutral pool we control. If the neutral run agrees with the original provider (within declared tolerances), we clear the provider. If it disagrees too often, we reject that package, penalize the provider, and immediately reassign the whole package elsewhere. We also produce a compact evidence bundle (inputs, outputs, diffs, and the randomness seeds) so anyone can understand and reproduce the ruling.

Design goals.

- **Fast:** end-to-end dispute time $\leq$ a few minutes; does not jeopardize the job deadline.
- **Deterministic:** fixed sampling rates, predeclared tolerances, no ad hoc overrides.
- **Auditable:** every decision comes with a machine-verifiable evidence bundle and Merkle proofs that tie back to on-chain roots (Components 0.5 and 0.6).

Data model (Aurora; Prisma additions). [language=sql] model Dispute id String @id @default(cuid()) jobId String @index packageId String @index @unique trigger String // 'dup$_{m}ismatch'|'canary_{f}ail'|'spo$
$10thresholdBpInt//mismatchtoleranceinbpofsampleditems(e.g., 50 => 0.5mismatchBpInt?//observedmismat$
$ownedpoolexecuted$

model DisputeItem id String @id @default(cuid()) disputeId String @index itemKey String @index // stable key from input manifest status String // 'queued'|'running'|'done' outcome String? // 'match'|'mismatch$_{w}ithin_{t}ol'|'mismatch_{o}ut_{o}f_{t}ol'diffUriString?//optionalcompactdiff$

Step A — Triggers (from QC). The dispute engine opens a `Dispute` when QC (Component 0.5) records any of:

- **Dup mismatch**: duplicated package outputs do not agree within tolerance.
- **Canary fail**: at least one canary item fails on the first pass.
- **Spot fail**: a spot re-exec on a non-duplicated package fails.

QC posts `POST /disputes/open` with {jobId, packageId, trigger}.

Step B — Sampling plan & timeline (normative v1).

- **Sample size:** $q = 10\%$ of the package's items, rounded up to a minimum of 100 items (or all if fewer than 1000 total). Stored as `sampleRateBp=1000`.
- **Selection:** seeded RNG derived from the job's QC seed (commit recorded in `QCPlan`); selection is independent of any provider choice.
- **Decision threshold:** if the fraction of *out-of-tolerance* mismatches on the sample exceeds θ (default $\theta = 0.5\%$, `thresholdBp=50`), we rule `provider_at_fault`; otherwise `provider_cleared`.
- **SLA:** neutral execution begins immediately; target completion time for the sample < 3 minutes for T1-sized packages so reassignment can still complete within the window.

Step C — Neutral pool (how we re-run).

1. **Environment capture:** use the *exact* `image_digest`, deterministic `command`, and the subset of inputs addressed by `itemKey` from the original manifest; fetch from Artifact Relay (Component 0.12).
2. **Isolation:** neutral nodes are QC-owned ProviderNodes with egress allowlists; no cross-talk with providers; secrets only allow read-access to the buyer's allowlisted buckets.
3. **Comparator:** same canonicalization/tolerance rules as QC (`bit_exact` or `fp_tolerant`); record outcome per item.

Step D — Decision rule & actions (package-scoped). Given n sampled items and m *out-of-tolerance* mismatches:

$$\widehat{\epsilon} = \frac{m}{n} \qquad \text{Decision} = \begin{cases} \texttt{provider_at_fault} & \text{if } \widehat{\epsilon} > \theta, \\ \texttt{provider_cleared} & \text{otherwise.} \end{cases}$$

- **If `provider_at_fault`:** mark the originating `Receipt.decision='rejected'`; POST `/lots/{id}/cancel` with reason `'qc_dispute_fail'`; scheduler (Component 0.4) *reassigns the whole package* immediately. Post a `ProviderTxn(kind='slash')` (Component 0.6).
- **If `provider_cleared`:** the package is `accepted`; no slash; record the mismatch rate and outcomes for transparency in the QC log.

Step E — Evidence packaging (for audits & slashing).

- **Contents (zip):**

1. `dispute.json`: trigger, seed derivation string, `sampleRateBp`, `thresholdBp`, $m, n, \widehat{\epsilon}$, decision.
2. `items.csv`: the sampled `itemKeys` and outcomes (match / mismatch_within_tol / mismatch_out_of_tol).
3. `env.json`: `image_digest`, `command`, neutral pool node metadata (tier/arch), timestamps.
4. `diffs/`: compact diffs for the first K offending items (schema-specific; capped).

- **Hashes:** every file is hashed; the bundle has a root hash added to the `qc_root` tree (Component 0.5) so any later alteration is detectable.
- **Availability:** buyer and the affected provider can download the bundle; a public "proof of decision" (hashes only) is shown in the job receipt bundle.

APIs (REST; internal where noted).

- `POST /disputes/open` [*internal*] → create a `Dispute`; returns the sampled item list.
- `POST /disputes/{packageId}/run` [*internal*] → start neutral execution; streams progress.
- `POST /disputes/{packageId}/decide` [*internal*] → finalize decision, store mismatch rate, emit actions (cancel/reassign or accept).
- `GET /disputes/{packageId}` → status, mismatch rate, evidence link (role-gated).

Acceptance tests (demo-grade).

1. **False alarm path (clear):** inject tiny numerical drift within tolerance in a duplicated package. Trigger opens; neutral run finds $\widehat{\epsilon} < \theta$; decision `provider_cleared`; package `accepted`; no slash.
2. **Real fault path (slash & reassign):** corrupt 1% of items beyond tolerance. Neutral run finds $\widehat{\epsilon} > \theta$; decision `provider_at_fault`; scheduler cancels & reassigns; slashing event recorded; reassigned package completes before deadline.
3. **Evidence integrity:** download bundle; verify file hashes and inclusion in `qc_root` using the `ngh-verify` CLI.

Parameters (v1 defaults, configurable).

Parameter	Default	Notes
Sample rate q	10% (1000 bp)	Min 100 items; raises cost mildly, strong detection.
Threshold θ	0.5% (50 bp)	Past this, rule at-fault.
Neutral pool size	2−5% of hour capacity	Shared across jobs; sized to keep disputes sub-3m.
Max diffs saved	$K = 100$	To cap bundle size; top by magnitude.

Security & privacy.

- **Seed hygiene:** dispute sampling seeds are derived from the QC plan seed; reveal strategy matches QC (commit–reveal v1.1).

- **Bucket access:** neutral workers receive short-lived, scope-limited credentials; outputs are written to QC-owned buckets.
- **Provider anonymity (optional):** public evidence bundles may redact provider identity; full bundle available to venue ops for slashing and to the provider for appeal.

Research tie-ins (quantifiable guarantees).

- **Detection power:** choose (q, θ) to achieve $\Pr(\text{detect} \mid \epsilon \geq \epsilon_0) \geq 1 - \beta$. With Hoeffding/Chernoff bounds on Bernoulli mismatches, we can publish (ϵ_0, β) pairs per schema.
- **Cost control:** measure neutral-pool overhead per hour; verify it stays within budget (target $< 3\%$ of total NGH).

Operator runbook (during incidents).

1. **Alert intake:** on `dup_mismatch` / `canary_fail` / `spot_fail`, confirm the dispute opened and neutral pool capacity is available.
2. **Watch timer:** if a dispute exceeds 3 minutes, consider temporarily bumping sample q down to 5% for this hour (log deviation in QC root).
3. **Postmortem:** for at-fault cases, review diffs and agent logs; if multiple packages from the same provider fault in a day, auto-quarantine their nodes.

Common pitfalls & cures.

- **Ambiguous numeric drift:** always rely on the canonicalization library and dual thresholds (rel_tol and ULP). For bit-exact packages, require matching `arch`.
- **Clock pressure:** disputes that can't finish quickly must not block reassignment; scheduler proceeds to reassign in parallel; dispute still determines slashing.
- **Seed leakage:** never log raw seeds before settlement; log only the hash-commit.

0.10 Component 10 — Canonicalization & Content Hashing Libraries (Per Output Schema)

This component provides the **single source of truth** for how outputs are turned into *comparable* byte strings and then *hashed* for receipts and QC. It ensures two runs that are *semantically the same*—even if item order differs or floating-point arithmetic jitters a few ULP—compare equal under our policy. All QC (Component 0.5), Disputes (Component 0.9), and Settlement (Component 0.6) call into this library.

Plain explanation (what this does). Jobs write results (tables, vectors, media). Before we check or pay, we normalize those results into a canonical form: sort records deterministically, round numbers in a documented way, use a fixed file encoding, and then compute a streaming Merkle hash. When we later compare two results, we compare their canonicalized forms with either **bit-exact** equality or **FP-tolerant** equality (small float differences allowed). Because everyone uses the same library, comparisons are consistent and auditable.

Outcomes (what "done" means).

- A versioned library `@ngh/canon` (Rust core with Node/TS bindings) exposing:
  - `canon::sort_stable(schema, records)` → `records'` (deterministic ordering)
  - `canon::round(schema, record)` → `record'` (numeric clamping/rounding)
  - `canon::serialize(schema, records')` → `bytes` (stable encoding)
  - `canon::merkle_stream(bytes|iterator)` → `root` (streaming SHA-256 Merkle)
  - `canon::equal(schema, a, b, mode)` → `bool` (`bit_exact` or `fp_tolerant`)
- Schema guides for `embeddings@1`, `tabular@1`, `video@1`: sort keys, rounding rules, and byte encodings.
- Fuzz/compat tests across two GPU families and two OS images showing false-reject/accept rates within targets.

Data shapes supported (v1).

- **Tabular** (`tabular@1`): rows of key/value columns; numerics (float32/64), ints, strings, booleans. Primary key(s) declared by the schema.
- **Vectors** (`embeddings@1`): JSONL of {`id`, `vector:[f32]`, `meta?`}.
- **Media** (`video@1`): per-item artifacts with sidecar JSON of metrics; canonicalization covers the sidecar and the per-chunk digests of media (we do *not* re-encode media).

Step A — Deterministic ordering.

1. **Declare sort keys per schema.**
   - `tabular@1`: lexicographic by the declared primary key tuple; if absent, by stable `id`.
   - `embeddings@1`: lexicographic by `id` (UTF-8, case-sensitive).
   - `video@1`: lexicographic by `item_id` then `chunk_index`.
2. **Stable sort** using ICU/bytewise order for strings (normalize to NFC; UTF-8 bytes) and numeric ascending for ints/floats; ties break by field order.

Step B — Numeric canonicalization (rounding & clamping).

1. **Float format and rounding.**
   - Internally keep IEEE-754 binary32/64 as produced; for canonical JSON/CSV, format with *round-half-to-even* to ≤ 9 significant digits for f32, ≤ 17 for f64.
   - **Denormals**: clamp subnormals to zero if $|x| < 2^{-149}$ (f32) or $< 2^{-1074}$ (f64); record a flag in `receipt.json` if clamping was applied.
   - **NaN/Inf**: disallowed by default. If a schema allows them, encode `"NaN"`, `"+Inf"`, `"-Inf"` strings, and require exact match even in FP-tolerant mode.
2. **Integer and boolean fields**: emit as base-10 ASCII without leading zeros; booleans as `true/false`.

Step C — Stable serialization (bytes to hash).

1. **Tabular & vectors**: *Canonical JSON Lines* (JCS-style): keys sorted ASC; no whitespace; UTF-8; newline-terminated per record. Example: `{"id":"a","v":[0.12,-0.7]}`
2. **Media**: do not transcode. Instead, compute per-chunk SHA-256 over the original bytes (HTTP range GETs), then serialize a JSON sidecar with `[{"chunk":0,"sha256":"0x.."}, ...]` and include media metadata (codec, duration).
3. **Parquet (optional)**: if users output Parquet, we *re-emit* a canonical JSONL by scanning the Parquet and applying Steps A–B, so hashing is always over the same wire format.

Step D — Streaming Merkle hashing (root for receipts).

1. **Leaf granularity**: chunk the canonical byte stream into fixed 1 MiB leaves (last leaf shorter); leaf_i = SHA256(bytes[i]).
2. **Tree**: pairwise hash node = SHA256($0x01 \parallel$ left $\parallel$ right) (prefix 0x01 to prevent collisions); single leaf $\Rightarrow$ root is that leaf.
3. **Ordering**: tree built over the canonical order (from Step A). We store `output_root` (hex) in the receipt and keep inclusion branches for proofs in the receipt bundle.

Step E — Equality modes for QC & disputes.

1. **Bit-exact**: byte-for-byte equality of canonical byte streams (or equality of Merkle roots if both used the same canon version).
2. **FP-tolerant**: elementwise check for numeric fields; non-numeric must be exact after canonicalization.

$$\text{rel_ok}(x, y) := \frac{|x - y|}{\max(\epsilon, |y|)} \leq 10^{-4}, \quad \text{ulp_ok}(x, y) := \text{ULP}(x, y) \leq 2.$$

   Both must hold. Here $\epsilon = 10^{-12}$ prevents division by near-zero. ULP distance is computed by reinterpreting floats as signed-magnitude integers and taking absolute difference.

Step F — Schema profiles (practical defaults).

- `embeddings@1`:
  - Sort by `id`; vectors compared elementwise; length must match exactly.
  - Suggested export: JSONL with `id:string`, `vector:[f32]`, optional `norm:f32`.
  - FP-tolerant recommended; bit-exact requires same CUDA arch and BLAS kernels.
- `tabular@1`:
  - Primary keys must be declared; duplicates rejected pre-hash.
  - String normalization: NFC; forbidden control chars; length cap (documented).
  - Mixed numeric types coerced to declared column type before canonicalization.

- `video@1`:
  - Compare sidecar metrics (fps, duration, checksum list) under bit-exact; FP-tolerant applies only to numeric metrics, not the media bytes.
  - Merkle roots are taken over sidecars; media chunks' SHA-256 included as data, not re-hashed again.

Step G — Packaging, versioning, and compatibility.

- **Version header**: every receipt includes `canon_version="canon-r1"` and `schema="embeddings@1"` so verifiers pick the right rules.
- **Bindings**: publish `@ngh/canon` (Node), `ngh-canon` (Rust crate). Deterministic builds; supply SBOM; sign releases.
- **Back-compat**: new canon versions must include a *down-cast* tool to re-emit old canonical bytes from new outputs when feasible.

Step H — APIs used by other components.

- QC/Dispute: `POST /canon/compare` with {`schema, mode, a_uri, b_uri`} → {`equal`, `stats`, `diff_uri?`}.
- Providers (optional): `POST /canon/hash` with {`schema, output_uri`} → {`output_root`}, to pre-compute roots client-side (not authoritative, but helpful).
- Tooling: `ngh-verify` CLI subcommand `canon hash|compare`.

Step I — Acceptance tests (demo-grade).

1. **Order invariance**: shuffle rows of a tabular output; canonical bytes identical; Merkle root unchanged.
2. **Float jitter**: add 1–2 ULP noise to 1% of an embedding vector; FP-tolerant comparison returns equal; bit-exact returns unequal.
3. **Parquet → JSONL**: export a Parquet table; re-emit canonical JSONL; repeated exports give byte-identical streams.
4. **Cross-platform**: run canonicalization on Linux and macOS images; byte outputs identical.

Step J — Security, integrity, and performance.

- **Streaming**: the Merkle builder consumes <64 MiB RAM regardless of output size; supports back-pressure (HTTP streams).
- **Hash choice**: SHA-256 (FIPS approved) with domain-separation prefix $0x01$ for inner nodes; leaves are raw SHA256(chunk).
- **Side-channel safety**: constant-time ULP comparison for floats; avoid locale-dependent formatting.
- **Reproducibility**: pin ICU/Unicode tables; vendor a minimal collation to avoid OS differences.

Research tie-ins (what to quantify next).

- **False decision rates**: estimate Pr(false-reject) and Pr(false-accept) on a gold corpus across two GPU architectures; target $\leq 10^{-6}$ and $\leq 10^{-8}$, respectively.
- **Tolerance stress**: sweep rel_tol and ULP thresholds to validate the chosen pair balances detection and stability under typical kernels (BLAS, conv).
- **Hash collision budget**: document effective collision probability with Merkle + SHA-256; provide guidance for v2 (e.g., SHA-512/256) if ever needed.

Operator runbook (during incidents).

1. If QC shows unexpected mismatches, first confirm both sides used the same `canon_version` and `schema`; re-hash locally with `ngh-verify`.
2. If denormal clamping flips a decision, record the clamp flag and consider pinning `arch` for that job (buyer guidance).
3. If Parquet readers disagree, enforce JSONL re-emit path and deprecate direct Parquet hashing for that schema.

Common pitfalls & cures.

- **Locale bugs**: never use default string sort—always ICU bytewise; normalize to NFC.
- **Hidden whitespace**: canonical JSONL is minified; any whitespace in values must be explicit (no accidental pretty-printing).
- **Mixed precisions**: if producers emit f16/bf16, up-cast to f32 before canonicalization and document in the schema.

0.11 Component 11 — Buyer Packager CLI & Preflight (Sizing, Canaries, Provenance)

This component gives Alice (and every buyer) a **single command-line tool** to turn a workload into sealed, schedulable *packages*. It (i) **sizes** packages so they meet milestones on the slowest allowed tier, (ii) **injects canaries** and builds manifests with **Merkle roots**, (iii) sets **determinism** and **capability requirements**, (iv) runs a **dry-run profiler** to catch bad sizes early, and (v) registers large artifacts with the **Artifact Relay** (Component 0.12). The output is a versioned `PackageDescriptor v1` index ready for submission to the Buyer Portal or API.

Plain explanation (what this does). You point the CLI at your container image and your dataset, choose how many parallel chunks you want, and it tells you: "Here's a safe way to split your job into ~ 10 NGH chunks that will start producing results within two minutes on the worst hardware you said you're willing to use." It adds a small set of known-answer inputs (canaries), records the exact bytes of your inputs so we can verify later, and writes one JSON file per package with all the knobs set (`requires.tier`, `min_vram_gb`, determinism mode, etc.). Finally, it uploads your big files to storage and registers them with the Relay so providers can fetch them over HTTP quickly.

Outcomes (what "done" means).

- A cross-platform CLI, `ngh-pack`, with subcommands: `init`, `size`, `canaries`, `manifest`, `dryrun`, `seal`, `relay register`, `validate`.
- A reproducible `package_index.jsonl` containing 100−1000 `PackageDescriptor v1` entries (see Component 0.12).
- Preflight checks that *refuse* to seal if milestone feasibility is unlikely on the slowest allowed tier, preventing "oversized" packages from reaching the scheduler.

Inputs & outputs (buyer-facing).

- **Inputs**: (a) container `image_ref` (OCI), (b) `command` (entrypoint + args), (c) dataset root(s) (URIs), (d) schema (`embeddings@1`, `tabular@1`, ...), (e) target tiers and attributes (`requires.*`), (f) desired parallelism N or target NGH per package.
- **Outputs**: (a) `package_index.jsonl` (≈ list of sealed packages), (b) per-package `input_manifest.parquet` with Merkle root, (c) `canaries.parquet`, (d) Relay registration receipts & checksums, (e) a `packager_report.html` explaining sizing, risk bands, and any warnings.

Local state (generated by the CLI). [language=json,caption=Packager workspace metadata (local only)] "job$_name$" : "$alice_embeddings_2026_02$", "$image_ref$" : "$oci://registry/alice/worker$: 1.4.2", "$schema$" : "$embeddings@1$", "$determinism$" : "$fp_tolerant$", "$tolerance$" : "rel_tol" : $1e-4$, "max_ulp" : 2, "r "$tier$" : "$T1$", "min_vram_gb" : 80, "$driver_stack$" : "$cuda12.4 + cuDNN9$", "$prefers$" : "$tier$" : "$T1$", "$dataset_uris$" : ["$s3://alice/ds/v4/$"], "$relay_bucket$" : "$https://relay.example.com/u/alice/$", "$target_ngh$" : 10.0, "$max_pkgs$" : 500, "$sizing_profile$" : "$auto_t12026q1$", "$canary_rate$" : 0.01

Step A — Initialize workspace & pin image provenance.

1. **Init:** `ngh-pack init` asks for job name, schema, determinism, requires/prefers, dataset URIs, and target NGH; it writes the workspace file shown above.
2. **Image pinning:** `ngh-pack seal-image` resolves `image_ref` to an immutable `image_digest` (`sha256:...`). It pulls the image locally and (optionally) verifies *Sigstore/COSIGN* signatures; if verification fails, the CLI aborts (see Security in Component **??**).
3. **SBOM & CVE gate (optional in demo):** extract an SBOM; warn on high-severity CVEs; record a provenance note into `packager_report.html`.

Step B — Sizing advisor (milestone-feasible packages). We want each package to reach first outputs ≤ 120 s and complete by the hour on **the slowest allowed tier**. The advisor chooses an item count W per package. It uses a simple throughput model:

$$T_1(W) \approx t_{\text{pull}} + t_{\text{prefetch}}(B, C, d) + t_{\text{warmup}} + \frac{w_0}{r_{\min}}, \quad T_{\text{done}}(W) \approx T_1(W) + \frac{W - w_0}{r_{\min}} + t_{\text{tail}},$$

where $r_{\min}$ is the p05 device throughput for the tier and schema (tokens/s, images/s). The CLI ships with *per-tier profiles* updated periodically from production telemetry; buyers can override if they have better local measurements.

1. **Fetch profile:** retrieve $\{r_{\min}, t_{\text{pull}}, t_{\text{prefetch}}(\cdot), t_{\text{tail}}\}$ for (`requires.tier`, schema). If `requires.tier` lists multiple tiers (e.g., T1 or T2), *use the slowest.*
2. **Compute $W^\star$:** choose W such that (i) $5 \le \widehat{\text{NGH}}(W) \le 15$, (ii) $T_1(W) \le 120$ s with 95% confidence, (iii) $T_{\text{done}}(W) \le$ deadline with 99% confidence. If no such W exists, `ngh-pack` refuses to proceed and explains why.
3. **Parallelism plan:** compute the number of packages $N = \min(\texttt{max_pkgs}, \lceil \text{TotalWork}/W^\star \rceil)$. *We never split a package later* (no platform-side splitting).

Step C — Inject canaries & build *input manifests* with Merkle roots.

1. **Canaries:** select $\sim 1\%$ of items per package as "known-answer" inputs. For each canary, store the expected output digest (from a local oracle run or provided by Alice). Save to `canaries.parquet`.
2. **Manifest:** list every item key and its byte-range(s) or object path; compute a *Merkle-of-chunks* over the input byte stream (HTTP range GETs) using the same tree rules as in Component 0.10. Emit `input_manifest.parquet` and record `input_merkle_root`.
3. **Stability checks:** ensure ordering is deterministic (sorted by item key); ensure per-package manifests are *disjoint* (no overlapping items across packages).

Step D — Dry-run profiler (catch obvious infeasibility).

1. **Local probe:** run the container on a small sample (e.g., 500 items) with the same `command`. Measure VRAM peak, per-item latency, disk read rate. If VRAM exceeds `min_vram_gb` or latency implies $T_1 > 120$ s even at profile rates, `ngh-pack` warns or refuses.
2. **Cloud sampler (optional):** execute the probe on one T1 node in our staging pool to validate cross-environment behavior. Store probe telemetry in `packager_report.html`.

Step E — Register large artifacts with the Artifact Relay.

1. **Sharding:** for ≥ 1 GiB artifacts (models, checkpoints), split into fixed-size chunks (e.g., 64 MiB), compute per-chunk SHA-256, and upload.
2. **Register:** `ngh-pack relay register {artifact}` publishes the list of chunks and checksums; receive a *Relay receipt* with a stable URL prefix. Providers can now HTTP range-GET deterministically (Component 0.12).

Step F — Seal & emit `PackageDescriptor v1`. For each package, write: [language=json] "package$_id$" : "$pkg_0$00123", "$image_digest$" : "$oci : //...@sha256 : ...$", "$command$" : ["$python$", "$worker.py$", "$--cfg$", "$/cfg.yaml$"], "$input_manifest_uri$" : "$https : //bucket/job123/pkg_0$00123.$manifest.parquet$", "$input_merkle$ "$0xabc123...$", "$expected_items$" : 100000, "$determinism$" : "$fp_tolerant$", "$tolerance$" : "rel_tol" : $1e-4$, "max_ulp" : 2 "$https : //bucket/job123/canaries.parquet$", "$output_schema$" : "$embeddings@1$", "$output_uri_prefix$" : "$s3 : //alice/out/job123/pkg_0$00123/", "$requires$" : "$tier$" : "$T1$", "$min_vram_gb$" : 80, "$driver_stack$" : "$cuda12.4 + c$ "$tier$" : "$T1$" Append each JSON line to `package_index.jsonl`. The CLI also signs the index (optional) and records a hash for integrity.

Step G — Validate & refuse bad configurations.

- **Milestone feasibility:** refuse to seal if the slowest allowed tier (from `requires.tier`) fails the $T_1 \leq 120\,\mathrm{s}$ or deadline tests at 95/99% confidence.
- **VRAM mismatch:** refuse if local probe shows VRAM peak would exceed `min_vram_gb`.
- **Missing canaries:** warn if canaries $< 0.5\%$; allow override with `-no-canary` flag (logged into the index).
- **Duplicate items:** refuse if any item key appears in more than one package.

Step H — UI affordances (Buyer Portal).

- **Drag-and-drop index:** upload `package_index.jsonl`; the portal runs the same validators server-side and shows a *readiness score*.
- **Sizing explainer:** a small chart from the CLI's profile, showing predicted T_1 and deadline distributions for T1/T2 if both were allowed.
- **Red flags:** badges for determinism mode, requires.tier, and any waived checks.

Acceptance tests (demo-grade).

1. **Happy path:** produce 120 packages with $\widehat{\mathrm{NGH}} \in [5, 15]$; manifests disjoint; canaries $\sim 1\%$; `package_index.jsonl` validates.
2. **Oversize refusal:** artificially increase W so $T_1 > 120$ s on T1; CLI refuses with a clear message and suggested $W^\star$.
3. **Duplicate detection:** inject the same item key in two packages; CLI errors out before sealing.
4. **Relay registration:** a 5 GiB checkpoint is chunked, uploaded, and registered; retry of one failed chunk succeeds; checksums match.

Research tie-ins (what we measure/improve).

- **P1 Sizing:** continuously calibrate $r_{\min}$ and t_{prefetch} from production; target >95% first-output success without reassignment (see Component 0.4).
- **P2 Canonicalization:** if FP-tolerant jobs show borderline drift, nudge buyers to pin `arch` or switch modes (Component 0.10).
- **P4 Normalization:** optionally run the microbenchmarks locally to estimate the device's q-multiplier and show an expected `paidNgh` per package.

Security & privacy.

- **Image provenance:** require digest pinning; optional Sigstore verification; record provenance in the index.

- **Data handling:** the CLI never uploads buyer data without explicit consent; `-upload` must be set. Manifests contain *hashes* and *URIs*, not raw data.
- **Secrets:** if the job needs storage credentials at runtime, store them outside the package; providers receive short-lived credentials from the platform at prepare time.

Operator runbook (support and telemetry).

1. **Profile updates:** publish a monthly *sizing profile* per schema/tier; the CLI auto-updates these via a signed manifest.
2. **Support triage:** when a buyer reports "prepare slow," review the packager report and confirm $t_{\text{pull}}/t_{\text{prefetch}}$ vs region; suggest caching or larger chunk sizes if needed.
3. **Backfill tool:** re-shard a job into smaller packages if the buyer insists—*as a new job*; we never edit existing sealed packages.

Common pitfalls & cures.

- **Too many tiers allowed:** if a buyer allows T1 or T2 but sizes for T1, milestone misses occur on T2. The CLI warns and either sizes for T2 or asks to pin to T1.
- **Hidden randomness:** unseeded data augmentations make canaries non-deterministic; the CLI checks for fixed seeds and warns if canary replay differs.
- **Path instability:** non-deterministic file discovery (e.g., glob order) leads to manifest drift; the CLI sorts and normalizes item keys to a stable form.

0.12 Component 12 — Artifact Relay & Distribution (HTTP-Only, Shards, Integrity, Prefetch)

This component makes large job artifacts (container images, models, checkpoints, input shards, canary tables) **fast to fetch, easy to resume, and safe to verify** over plain HTTP. It is the backbone that lets providers meet the **first-output** ≤ 2 **minutes** milestone without any peer-to-peer tricks. The Relay is deliberately simple: content-addressed files, fixed-size chunks with checksums, edge caching, and a few APIs to *register*, *prefetch*, and *prove* integrity.

Plain explanation (what this does). Buyers upload big files once. The Relay cuts each file into equal chunks, computes a hash per chunk, and publishes stable HTTP URLs. Providers can then "range-GET" the exact chunks they need, verify the hash as they read, and resume if a transfer drops. Right before a job hour starts, the platform "prewarms" the most important chunks to nearby caches so the first few minutes are quick even if storage is far away.

Outcomes (what "done" means).

- A **content-addressed registry** of artifacts with **chunk manifests** and a single *root digest* per artifact.
- A **registration API** used by the Packager CLI (Component 0.11) that returns stable, cacheable URLs.

- Provider-facing **download rules**: range requests, per-chunk verification, resume, multi-origin fallback, rate limits.
- A **prewarm service** the scheduler calls at $T-3$ minutes to stage "first-chunk" bytes near providers.

Data model (Aurora; Relay catalog). [language=sql] model Artifact id String @id @default(cuid()) ownerAccount String @index name String @index – human label bytesTotal BigInt chunkSize Int – e.g., 64 MiB chunks Int rootSha256 String @index uriPrefix String – https://relay/p/id/c/ createdAt DateTime @default(now()) status String – 'registered'|'verified'|'retired'

model ArtifactChunk id String @id @default(cuid()) artifactId String @index index Int @index – 0..(chunks-1) size Int – last chunk may be smaller sha256 String uri String – stable path for GET verified Boolean @default(false) – optional server-side sampling

model ArtifactReceipt id String @id @default(cuid()) artifactId String @index ownerProof String – signature of (rootSha256, bytesTotal, chunkSize) canon String – canon version for hashing (see Comp. 0.10) createdAt DateTime @default(now())

Step A — Content addressing, chunking, and stable URLs.

1. **Chunking:** split the file into fixed C byte chunks (default C=64 MiB). The last chunk may be shorter.
2. **Chunk hashes:** $h_i = \mathrm{SHA256}(\mathrm{chunk}_i)$ for $i = 0, \ldots, k-1$.
3. **Root digest (Merkle-of-chunks):** build a binary tree over $\{h_i\}$ with node hash

$$H = \mathrm{SHA256}(0x01 \parallel H_L \parallel H_R), \quad \text{leaf } H = \mathrm{SHA256}(\mathrm{chunk}_i).$$

   Store the root as `rootSha256`. (Same conventions as Component 0.10.)
4. **Stable paths:** `/p/{artifactId}/c/{index}-{sha256}.bin`. Providers can safely cache keyed by full path (hash included).

Step B — Register an artifact (CLI ↔ Relay).

1. **Client-side prep (CLI):** compute $\{h_i\}$, the Merkle root, and a *manifest* (JSON) of chunk sizes and hashes.
2. **Register:** `POST /relay/register` with {`name, bytesTotal, chunkSize, rootSha256, chunks:[{i,siz` The Relay allocates `artifactId`, creates `Artifact`/`ArtifactChunk` rows, and returns upload URLs (S3 pre-signed PUT or multipart).
3. **Upload & verify:** the CLI uploads all chunks in parallel. The Relay samples s random chunks to *recompute* SHA256 server-side; if all pass, set `status='verified'` and issue an `ArtifactReceipt` signed by the server.

Step C — Provider download rules (fast, resumable, safe).

- **Range GETs & resume:** providers always request by exact chunk URL; use `Range` and validate `ETag`/`If-Range`.

- **Per-chunk verification:** after reading a chunk, compute SHA256 and compare with the URL's hash; on mismatch, *discard* and try the mirror URL (see fallback).
- **Fallback mirrors:** each chunk URL maps to two origins (e.g., S3 in region + replicated bucket or CDN back-origin). The agent tries primary, then secondary.
- **Backpressure & timeouts:** default read timeout $T_r = 10$s per MiB; exponential backoff with jitter; ≤ 4 concurrent chunk streams per artifact unless prefetch dictates otherwise.
- **Allowlists & quotas:** provider runtime only fetches from Relay origins in an allowlist; per-package byte quotas prevent runaway download.

Step D — Edge caching & prewarm for first-output ≤ 120s.

1. **Edge:** put `/p/*` behind CDN (e.g., CloudFront). Cache key = full path; TTL = 7 days; `Cache-Control: public,immutable`.
2. **Prewarm plan:** at $T-3$ minutes the scheduler calls `POST /relay/prewarm` with a list of (artifactId, firstNChunks) near each region/PoP used by assigned providers.
3. **Warm set size:** choose first d chunks per artifact so that the total warm bytes satisfy

$$t_{\text{pull}}(d \cdot C) + t_{\text{prefetch}}(d) + t_{\text{warmup}} \leq 30\,\text{s},$$

   leaving time for w_0 items to compute so first outputs appear within 120s. Defaults: C=64 MiB, $d \in [2, 8]$ depending on artifact mix and measured egress.
4. **PoP selection:** prewarm to the CDN edge nearest to each provider region (GeoIP or explicit mapping).

Step E — Tying manifests to packages (integrity chain).

- Packager emits `input_manifest.parquet` with item keys and byte ranges; it also records `rootSha256` for each referenced artifact.
- Provider runtime verifies that the fetched chunk hashes match both the URL suffix and the manifest's per-chunk entries; *any* mismatch fails prepare.
- The package *receipt* includes the `input_merkle_root`; this binds execution to immutable inputs (see Components 0.5, 0.6).

Step F — Security, privacy, and access control.

- **Signed URLs (optional):** for private artifacts, use time-limited, scope-limited pre-signed GET/HEAD; packager/portal grants providers short-lived tokens at prepare.
- **No P2P:** only HTTPS to Relay origins; egress allowlists in provider sandboxes (Component **??**).
- **Logging:** access logs per chunk (accountId, node fingerprint, bytes, latency) for performance and abuse detection.

Step G — APIs (Relay service).

- `POST /relay/register` → create `Artifact` & chunk rows; return upload URLs and `uriPrefix`.
- `POST /relay/verify` → (server-side) sample verify chunks and set `status='verified'`; issue `ArtifactReceipt`.
- `GET /relay/manifest/{artifactId}` → JSON chunk list with sizes, hashes, and URLs (public if artifact is public).
- `POST /relay/prewarm` → accept a list of (artifactId, firstNChunks, region) and trigger CDN prefetch or synthetic GETs.
- `GET /relay/health` → origin and CDN health, cache hit rate by region.

Step H — Frontend (Portal views).

- **Artifacts list:** name, size, root hash, verification status; copy-paste `uriPrefix`.
- **Heatmap:** cache hit-ratio by region for selected artifacts; "prewarm now" button (ops-only).
- **Integrity badge:** green when server sampling passed and chunk URLs match catalog; red if any chunk hash mismatch was reported by providers.

Step I — Acceptance tests (demo-grade).

1. **Corruption detection:** flip one byte in an uploaded chunk; server sampling or provider download fails verification; fallback mirror succeeds.
2. **Resume:** interrupt a large download at 60%; agent resumes and completes; total bytes <110% of file size (bound on retries).
3. **Cold vs warm:** measure first byte latency for first d chunks with and without prewarm; warm path reduces t_{pull} by $\geq 50\%$.
4. **Manifest check:** agent refuses to run if `rootSha256` in manifest doesn't match registered artifact; prepare status stays "not ready."

Step J — Performance SLOs (v1 targets).

Metric	Target
Edge hit ratio (warm set)	$\geq 80\%$ in provider region during $T{-}3$ to $T_0{+}5$ min
First byte (warm chunk)	< 200 ms p50, < 600 ms p95
Prefetch completion	$> 95\%$ of requested warm chunks cached before T_0
End-to-end integrity	$< 10^{-12}$ undetected corruption probability per job

Step K — Operator runbook.

1. **Before busy hours:** review prewarm queue; ensure replication across origins is healthy; purge stale artifacts flagged as "retired."

2. **On gate close:** confirm prewarm batches scheduled for all assigned packages' artifacts; watch cache heatmap.
3. **During execute:** alert on cache miss spikes or regional origin slowness; temporarily increase d (warm depth) if needed.
4. **After hour:** reconcile access logs vs assigned packages; investigate any hash mismatch reports.

Research tie-ins (what we quantify next).

- **Prefetch policy:** choose C and d to minimize expected T_1 under bandwidth/RTT distributions; publish a simple rule of thumb (e.g., $d = \lceil 2 + \frac{t_{\text{warmup}}}{C/B} \rceil$).
- **Multi-origin control:** model failover cost vs. dual-write cost; tune sampling rate s for server verification to catch origin corruption with high confidence.
- **Regional replication:** decide which artifacts to replicate proactively based on nomination curves and historical reuse.

Common pitfalls & cures.

- **Mismatched manifests:** if a buyer re-uploads an artifact but forgets to re-seal packages, the root hash changes. The provider agent refuses to run; the portal should show "re-seal needed."
- **Over-prefetching:** too large d clogs edges; watch cache hit/miss and throttle d per hour.
- **Hidden redirects:** never rely on 302s to third-party storage; embed final chunk URLs in the catalog so caches stay hot and integrity checks remain local.

0.13 Component 13 — Regionalization & Multi-Region Keys (Locality, Latency, Compliance)

This component makes the marketplace *regional* by construction. Every financial and operational object (vouchers, indices, nominations, placements, QC, settlement) is keyed by **region** and **hour (UTC)**, so compute bought for `us-east, 14:00-15:00` is *delivered in us-east during that exact UTC hour*. The goals are: (i) keep data where it belongs (locality & compliance), (ii) keep start-up fast (low latency; caches near providers), and (iii) isolate failures (a problem in one region must not disrupt others).

Plain explanation (what this does). Think of each region as its own "mini–exchange + datacenter." Providers join a region; buyers pick the region close to their data. Vouchers are stamped with that region and can only be used there. The scheduler, artifact caches, QC pool, and settlement for that hour all run *in that same region*. If you need two regions, you create two jobs and buy two sets of vouchers. If one region is unhealthy, the others keep working.

Outcomes (what "done" means).

- A **single source of truth** for region metadata and *region-scoped* keys for vouchers, indices, nominations, and placements.

- A **sharded control plane**: per-region schedulers, QC pools, and Relay points-of-presence (PoPs); a thin global directory just routes requests to the right shard.
- **UI correctness**: buyers see hours in local time *and* UTC; all on-chain and database timestamps use UTC to avoid daylight-saving confusion.

Data model (Aurora; minimal additions). [language=sql] – Region catalog and routing model RegionConfig id String @id // 'us-east', 'eu-west', ... displayName String timezone String // e.g., 'America/New$_York'relayPopUrlString//preferredRelayPOPforregionqcPoolLabelString//neutralpoolIDinre$
$chainanchoringenabledBoolean@default(true)createdAtDateTime@default(now())$

– All regionalized instruments and books share this "key" shape model HourKey id String @id // cuid region String @index isoHour DateTime @index // start of hour, UTC (e.g., 2026-01-18T19:00:00Z) sla String @index // 'standard' tier String @index // 'T1'|'T2'|'T3' UNIQUE(region, isoHour, sla, tier)

– Voucher escrow per (region, hour, SLA, tier) model VoucherEscrow id String @id @default(cuid()) jobId String @index hourKeyId String @index deposited BigInt // microNGH consumed BigInt // microNGH createdAt DateTime @default(now())

– Scheduler shard assignment for a job/hour model RegionPlacementPlan id String @id @default(cuid()) jobId String @index hourKeyId String @index schedulerUrl String state String // 'draft'|'frozen'|'executing'|'final' createdAt DateTime @default(now()) updatedAt DateTime @updatedAt

– Region-scoped index print (spot VWAP) model HourlyIndexPrint id String @id @default(cuid()) hourKeyId String @index vwapPriceBps Int // price * 10,000 (basis points), or a fixed unit in demo tradesUsed Int depthMetric Int createdAt DateTime @default(now())

Step A — Keys and time rules (authoritative).

1. **Hour boundaries:** all job hours are defined in **UTC**. The UI shows both local time and UTC, but the `isoHour` we store is the UTC start timestamp with minutes/seconds zeroed.
2. **Voucher key:** $\langle$`region`, `isoHour`, `sla`, `tier`$\rangle$. A voucher minted for `eu-west, 18:00-19:00Z, Tier 1, Standard` *cannot* be used in `us-east` or at a different hour.
3. **Package binding:** a job submitted for a given region must produce a package index whose `requires` do not contradict the region policy (e.g., data residency).

Step B — Sharded control plane (one shard per region).

- **Schedulers:** each region has an independent scheduler instance and queue (Component 0.4). At gate close, the `RegionPlacementPlan` for that region flips to `'frozen'` and only that scheduler can mutate it.
- **QC pools:** each region has its own QC neutral pool and random seed stream (Component 0.5). No cross-region duplication in v1.
- **Relay PoPs:** artifact URLs resolve to the region's nearest edge (Component 0.12); prewarm only touches that edge.

- **Settlement L2:** anchoring and settlement calls use the region's configured L2 (e.g., `us-east` → L2-A, `eu-west` → L2-B) to keep legal entities and fees clean (Component 0.6).

Step C — Providers join a region; jobs pick one.

- **Provider region:** each `ProviderNode` (Component 0.8) declares exactly one `region`. A node can only be nominated and assigned within its `region`.
- **Job region:** buyers choose a single `region` per job. Multi-region workflows are modeled as multiple jobs (one per region).
- **Data locality guidance:** the Buyer Portal warns if the job's input buckets are remote (cross-region) because it will hurt the $\leq$ 120s first-output milestone.

Step D — Compliance flags and residency fences.

- **Residency flag:** a job can set `requires.residency='strict'`; then the platform forbids fetching inputs from buckets outside `region` and rejects provider nodes whose network routes are unknown.
- **Secrets per region:** storage credentials are issued by a region-local secrets service; tokens are scoped to region-local endpoints.
- **Logs & PII:** provider logs and QC evidence stay in region-local buckets; cross-region analytics use anonymized aggregates.

Step E — Cross-region voucher transfers (not in v1).

- **v1 policy:** *no transfers.* A voucher's region is immutable; secondary trading exists *within* a region only.
- **v2 sketch (admin bridge):** if ever needed, an on-chain "bridge" could retire a voucher in region A and mint a new one in region B at a conversion based on published indices. Out of scope for demo; requires manipulation-resistant indices (Component 0.7).

APIs (global directory & region shards).

- `GET /regions` → list of enabled regions with `timezone`, `relayPopUrl`, `qcPoolLabel`, `settlementL2`.
- `GET /regions/{id}/hours?from=&to=` → upcoming UTC hours with market depth (nominations) and last index prints.
- `POST /jobs` → requires `region`; server returns the region's `schedulerUrl` for feasibility and placement.
- `POST /nominations` → region-scoped; only accepts `accountId` with nodes in that region.

Acceptance tests (demo-grade).

1. **Time correctness:** schedule a job for `us-east` "2–3 pm local." The portal converts to `isoHour` = 19:00–20:00Z (Jan). Gate-close occurs at 18:55Z; placement happens only in `us-east`.

2. **Regional isolation:** simulate a `eu-west` Relay outage; `us-east` jobs run unaffected. QC and settlement events for `us-east` continue on its L2.
3. **Residency fence:** set `requires.residency='strict'` and use an `s3://` bucket in another region; prepare fails with a clear error; succeed after moving data local.
4. **Index scope:** the hourly spot index for `us-east, T1` does not include any `eu-west` prints; settlement picks the correct regional index when needed.

SLOs & operational metrics (per region).

Metric	Target
Gate-close latency (control)	$<$ 1 s p95 from trigger to frozen plan
Cross-region blast radius	No impact outside failing region (qualitative drill)
Relay first-byte (warm)	$<$ 200 ms p50 within region (see Comp. 0.12)
QC dispute time	$<$ 3 min p95 within region (see Comp. 0.9)
On-chain anchor delay	$<$ 60 s p95 on region's L2 (see Comp. 0.6)

Security & resilience (per region).

- **Least privilege:** providers in `us-east` receive credentials that cannot read `eu-west` buckets; Relay PoP ACLs enforced by origin.
- **Keys/KMS:** secrets and KMS keys are region-local; audit logs prove no cross-region secrets exfiltration.
- **Failover stance:** we *do not* auto-migrate jobs to other regions in v1; instead, we cancel and refund if a region cannot meet SLA.

Research tie-ins (what to quantify next).

- **Regional spot indices:** microstructure design to prevent thin-market manipulation in small regions (see Component 0.7).
- **Data-locality pricing:** quantify the premium/discount for region-local data vs cross-region I/O on first-output success probability and adjust buyer guidance.
- **Shard controller:** formalize a controller that auto-tunes Relay prewarm depth and QC pool size per region by observing hour-by-hour demand.

Operator runbook (daily cadence).

1. **Pre-hour checks:** for each region, verify scheduler health, QC pool capacity, Relay cache heat, and on-chain L2 status.
2. **Gate close:** ensure only the region's scheduler flips plans to `'frozen'`; watch for clock skew (all shards must use NTP).
3. **After hour:** reconcile accepted NGH vs consumed vouchers *per region*; publish index prints; archive logs locally.

Common pitfalls & cures.

- **DST confusion:** always show "Local (Region TZ) *and* UTC" in the UI; store only UTC. Add a banner during DST transitions.
- **Mixed-region artifacts:** buyers sometimes point to a bucket in another region; the portal should flag this and estimate the first-output penalty.
- **Cross-region dependencies:** avoid letting a region's scheduler call another region's QC/Relay directly; use the global directory to fetch region endpoints and keep calls local.

0.14 Component 14 — Security & Supply Chain (Image Admission, SBOM, CVE Gates)

This component keeps the platform safe while still letting buyers run their own code. In plain terms: before any container image runs on any provider, we *prove what it is* (signature & digest), *know what's inside* (SBOM: a bill of materials), and *check it for known vulnerabilities* (CVE scan). If something looks risky, we place it in a **quarantine lane** for review instead of letting it touch providers. All of this is done with minimal delay so the ≤ 2 minute first-output milestone remains achievable.

Outcomes (what "done" means).

- **Admission policy** that blocks unpinned or unsigned images and fails builds with high-severity CVEs unless a time-bound waiver exists.
- **SBOM everywhere:** every image has an attached SPDX/CycloneDX SBOM; we store the SBOM digest and link it to the image digest.
- **Fast path + quarantine:** clean images get a *green badge* and are cache-approved for fast pulls; flagged images are routed to quarantine with clear remediation steps.
- **Runtime hardening** on provider hosts: non-root containers, read-only root FS, egress allowlist, GPU access scoped to the assigned lot only.

Plain explanation (why this matters). Providers will run buyer-supplied containers. Without guardrails, a bad image could try to exfiltrate data, mine crypto, or just crash machines. Our checks make sure the image is exactly the one the buyer intended (signature & digest), list the libraries it carries (SBOM), and block images with known critical bugs (CVE scan). We also run containers with the minimum privileges needed, and we only allow network access to the buyer's storage and the Artifact Relay.

Data model (Aurora; minimal). [language=sql] model ImageAdmission id String @id @default(cuid()) imageRef String // buyer-submitted ref (e.g., oci://repo/worker:1.2) imageDigest String @unique // immutable sha256:... submittedBy String @index // buyer account id signedBy String? // identity from cosign (OIDC email/issuer) sbomUri String? // stored SBOM (SPDX/CycloneDX) sbomSha256 String? // SBOM content hash cveMaxSeverity String // 'none'|'low'|'medium'|'high'|'critical' cveCountHigh Int cveCountCrit Int policyStatus String //

'approved'|'quarantine'|'rejected' reason String? // why quarantined/rejected createdAt DateTime @default(now()) updatedAt DateTime @updatedAt

model CveWaiver id String @id @default(cuid()) imageDigest String @index cveId String // e.g., CVE-2024-12345 expiresAt DateTime // hard expiry; past this, admission fails justification String approvedBy String // ops user id createdAt DateTime @default(now())

Step A — Submission-time checks (fast and cacheable).

1. **Pin the image:** resolve `imageRef` to immutable `imageDigest` and *require* digest pinning in the PackageDescriptor (Component 0.11 enforces this).
2. **Signature verify (cosign):** verify a Sigstore/COSIGN signature against an allowed issuer (e.g., GitHub Actions OIDC) or a buyer-owned key. Record `signedBy`. If signature is missing or invalid, set `policyStatus='quarantine'` with a clear reason.
3. **Generate SBOM:** run *Syft* (or equivalent) to produce an SPDX or CycloneDX SBOM; store at `sbomUri` with `sbomSha256`.
4. **CVE scan:** run *Grype/Trivy* against the SBOM and image layers; record counts and highest severity. If any `critical` findings without waivers, *reject*; if `high` exist, *quarantine* unless an approved `CveWaiver` covers them. Otherwise, *approve*.
5. **Cache approvals:** on `approved`, the pull-through cache (ECR/registry mirror) marks the digest as *pullable* by providers to avoid re-scanning at prepare time.

Step B — Prepare-time gates (provider runtime).

1. **Digest-only pulls:** provider agent pulls the `imageDigest` (never a tag). If the digest is not `approved`, prepare fails with an actionable message.
2. **Signature attest:** provider validates the same cosign signature (public keys are distributed with the agent) and posts an attestation hash with `/lots/{id}/prepare_ready`.
3. **Sandbox profile:** containers run non-root, `readOnlyRootFilesystem=true`, dropped Linux caps (no `SYS_ADMIN` etc.), with seccomp/apparmor. Filesystem mounts are read-only except a small `/tmp`. GPU access is limited to the assigned device.
4. **Egress allowlist:** only Artifact Relay, buyer's allowlisted buckets, and the platform's control plane are reachable; all other egress is blocked.

Step C — Quarantine lane & waivers.

- **Quarantine flow:** flagged images get a portal card with the exact CVEs, affected packages, and remediation tips (upgrade base image, pin patched libs). Buyers may request a `CveWaiver` for *high*-severity findings if the code path is non-exploitable for their workload; waivers *expire* automatically.
- **Auto-retry:** when a new image digest is submitted (patched), the previous packages can be re-sealed automatically to reference the patched digest (buyer opt-in).

Step D — Runtime hardening (default profiles).

- **Non-root user**: UID/GID remap; disallow `setuid` binaries.
- **Resource limits**: cgroups on CPU/mem/IO; GPU-exclusive assignment; `ulimit` caps for files/processes.
- **Filesystem**: read-only root FS; writeable `/work` bounded by quota; no hostPath mounts; `noexec,nodev,nosuid` where applicable.
- **Secrets**: short-lived credentials mounted via tmpfs; rotated per package; never baked into images.

APIs (Portal & internal).

- `POST /admission/images/submit` → {`imageRef`} ⇒ returns `imageDigest`; enqueues SBOM & CVE scan; responds with provisional status.
- `GET /admission/images/{digest}` → current `policyStatus`, signature info, SBOM, CVE tallies.
- `POST /admission/waivers` → request/approve a `CveWaiver` (ops-only approval).
- `GET /admission/report?jobId=` → per-job admission summary (green/yellow/red) for all referenced images.

Acceptance tests (demo-grade).

1. **Unsigned image:** submit an image without a cosign signature; status `quarantine`; prepare fails with "unsigned" message.
2. **Critical CVE:** image with a critical vulnerability is `rejected`; cannot be used until patched; re-submit with patched digest leads to `approved`.
3. **Waiver path:** image with a single high CVE gets a time-bound waiver; packages run; after expiry, admission flips back to `quarantine`.
4. **Runtime policy:** attempt to open a disallowed egress URL; provider sandbox blocks and logs; job still completes with allowlisted endpoints.

Performance & SLOs (targets).

Metric	Target
Initial scan time (SBOM + CVE)	$\leq$ 60 s p95 (cached layers help)
Admission cache hit (reused digests)	$\geq$ 90% for popular bases
Prepare overhead on provider	$<$ 150 ms p95 beyond image pull time
Waiver turnaround (ops)	$<$ 4 h during business hours

Research tie-ins (next-level rigor).

- **Provenance scoring:** rank images by supply-chain trust (SLSA level, signature issuer, reproducible build status) and surface risk scores to buyers.
- **Policy tuning:** estimate break-even of strict vs. lenient CVE thresholds on incident probability and developer friction; calibrate defaults with data.
- **Reproducible builds:** optional "rebuild and compare" for high-stakes jobs: rebuild in a hermetic environment and compare the digest to catch substitution attacks.

Operator runbook (daily).

1. Review `quarantine` queue; nudge buyers with remediation steps; expire stale waivers.
2. Spot-audit provider logs for sandbox escapes (should be none); rotate provider agent keys periodically.
3. Keep CVE feeds and scanner databases fresh; verify scanners themselves with checksums/signatures.

Common pitfalls & cures.

- **Tag drift:** never admit `:latest`; require digest pinning and show the exact digest in the portal.
- **Overscanning:** avoid re-scanning the same digest on every submit; cache by digest and SBOM hash.
- **Privilege creep:** enforce policy-as-code (OPA/Rego) for runtime profiles; block any request for privileged mode.

1 CADO-NFS Integration — End-to-End Components (C1–C12)

This section isolates *everything* required to run **CADO-NFS** on our marketplace: how we split the workload across GPUs, which parts must stay on a single provider, how we package and verify results, and how we normalize performance for fair payment. Each component includes plain-language goals, concrete deliverables, and acceptance tests. Where the base platform already provides functionality, we specialize it for CADO-NFS.

C1 — Workload Decomposition & Success Criteria

Goal (simple): break CADO-NFS into phases we can schedule safely without cross-provider collectives. **Phases:**

1. **Polynomial selection (PS)** — small, CPU-bound; single package (optional precomputation).
2. **Relation collection / sieving (RC)** — embarrassingly parallel; *many* packages; GPU-accelerated sieving/filtering allowed.
3. **Filtering & matrix assembly (FA)** — builds the sparse GF(2) matrix; *can* be split by blocks but final stitch must be consistent.
4. **Linear algebra (LA)** — Block Lanczos/Wiedemann over GF(2) on a large sparse matrix; *single-provider package*, ideally multi-GPU with NVLink for speed.
5. **Square root & factor check (SQ)** — light, deterministic; single package.

Non-negotiables: no WAN collectives; a package runs on exactly one provider; LA is one big package on a provider that meets VRAM/NVLink constraints. **Success:** we factor a target N with bit-exact receipts, within the allotted window, and every accepted NGH ties to vouchers 1:1.

C2 — GPU Acceleration Plan (Kernels, Libraries, Determinism)

Goal (simple): use GPUs where it helps most while remaining *deterministic*. **Plan:**

- **Sieving (RC):** implement/port GPU kernels for lattice/line sieving hotspots (trial division and special-q loops). Use CUDA/HIP backends; expose knobs (`q_range`, `interval`, `bucket_size`) via package config. Determinism: fixed PRNG seeds; exact integer arithmetic.
- **Filtering/assembly (FA):** mostly CPU but enable GPU-assisted duplicate removal and coalescing; outputs must be identical byte-for-byte given same inputs.
- **Linear algebra (LA):** bit-sliced GF(2) SpMV on GPUs; Block Lanczos core with checkpointing every K iterations. Layout: CSR/ELLP hybrid with 64-bit word packing; warp-level XOR/popcount. Determinism: fixed reduction order per warp/block; avoid atomics with undefined order.

Deliverables: a containerized worker exposing `-phase={RC,FA,LA,SQ}` with CUDA/HIP builds; reproducibility tests across two SM architectures (bit-exact).

C3 — Package Types & Output Schemas (NFS-Specific)

Goal (simple): define *what each package produces* and how we compare it. **Schemas (canonical forms):**

- `nfs_relations@1`: JSONL; fields {`rid`, `a`, `b`, `q`, `primes:[int]`, `sign`}; sort by (`a`, `b`, `q`); integers exact; no floats.
- `nfs_matrix@1`: header (dim, nnz); blocks as CSR with row/col ascending; per-block SHA-256; top-level Merkle root over blocks.
- `nfs_deps@1`: list of dependency vector IDs (bitstrings) in lexicographic order; exact bits.
- `nfs_factors@1`: big-int limbs (base 2^{64}); canonical representation (no leading zero limbs); includes N, factors, and a proof check record.

Determinism: *bit-exact* for all `nfs_*` schemas. **Canaries:** tiny N (128–256-bit) with known factors; included in RC/FA/LA pipelines to sanity-check.

C4 — Splitting & Parallelization Strategy (Across GPUs, Safely)

Goal (simple): maximize parallelism in phases that allow it; confine global-dependence phases to one provider. **Strategy:**

1. **RC (many packages):** split by (`special-q`, `interval`) ranges. Each package: fetch polynomial, run sieving for assigned interval, dedup locally, emit `nfs_relations@1`. First outputs (counts) appear $\leq$ 120s.
2. **FA (moderate packages):** optional multi-package block assembly: each package assembles a disjoint set of rows/columns and emits `nfs_matrix@1` blocks + a *block index*. A *final FA-stitch package* merges block indices deterministically (still single provider).
3. **LA (single provider):** one package consumes the final matrix and runs Block Lanczos to completion; within that provider, allow *multi-GPU* if available (NVLink required). This respects "no cross-provider collectives."
4. **SQ (single provider):** uses LA outputs; emits `nfs_factors@1`.

Provider constraints:

- RC/FA: Tier-2 OK; modest VRAM; high disk read MB/s helps.
- LA: Tier-1 with $\geq$ 80GB VRAM, `nvlink_required=true`, `driver_stack=cuda12.4+`; optionally `gpus_per_host` ≥ 4.

C5 — Packager Plugin `ngh-pack cado` (Sealed Jobs)

Goal (simple): one command turns CADO-NFS inputs into a ready-to-submit package index. **What it does:**

1. **Detect phase** and dataset (polynomials, parameter files).
2. **RC sharding**: compute (`special-q`, `interval`) grid; size shards to ~ 10 NGH with $T_{\text{first-output}} \leq$ 120s on the slowest allowed tier.
3. **Inject canaries** and build `input_manifest` Merkle roots; register large tables with the Relay.
4. **Emit descriptors** with `requires` (tier, vram, nvlink, driver) and `determinism=bit_exact`.
5. **FA/LA configs**: compute block maps and LA checkpoint cadence (every K iterations) to support disputes.

Output: `package_index.jsonl` with RC*, FA*, (FA-stitch), LA, SQ entries.

C6 — Benchmarks & Normalization (q-Multiplier for NFS)

Goal (simple): pay fairly across devices; make vouchers hedgeable inside tiers. **Microbenchmarks:**

- **RC-bench:** validated relations/sec on a fixed (`q_range`, `interval`) and polynomial; the score excludes I/O waits.
- **LA-bench:** GF(2) SpMV iterations/sec on a *canned* matrix (dimension and nnz representative of demo); includes reduction/orthogonalization costs.

Normalization: $q = \frac{\text{baseline_score}}{\text{device_score}}$ per schema; `paid_ngh` = `gpu_time_hours_raw` $\times q$. Publish *tier caps* for q to bound basis risk.

C7 — Sizing & Milestone Feasibility (RC/FA/LA)

Goal (simple): every package meets "first output $\leq$ 120s" and deadline, especially on the slowest allowed tier. **Rules of thumb (CLI-enforced):**

$$T_1 \approx t_{\text{pull}} + t_{\text{prefetch}}(d, C) + t_{\text{warmup}} + \frac{w_0}{r_{\min}}, \quad T_{\text{done}} \approx T_1 + \frac{W - w_0}{r_{\min}} + t_{\text{tail}}.$$

- **RC**: choose interval length W so that first verified relation arrives by T_1; use ≥ 2 warmed chunks at the Relay (Comp. 0.12).
- **FA**: ensure each block assembly fits memory and emits its first block metadata $\leq$ 120s.
- **LA**: pre-stage matrix header + first K CSR tiles; pick K so one SpMV finishes $\leq$ 60s and a checkpoint is written by $\leq$ 120s.

C8 — Relay Layout & Checkpoints (NFS Artifacts)

Goal (simple): make big NFS files fetch fast and verifiable. **Artifacts:**

- **Polynomials/params**: small; replicate broadly; immutable digests.
- **RC outputs**: many medium JSONL files; 64 MiB chunking; per-chunk SHA-256.
- **Matrix blocks**: CSR tiles with sidecar JSON (dim, nnz, offsets); per-block SHA-256; top-level Merkle.
- **LA checkpoints**: vectors and iteration state; index by iteration number; hashed sidecars.

Prewarm: first $d \in [2, 8]$ chunks for each active artifact near assigned providers to keep t_{pull} small.

C9 — Provider Runtime Profile (Resources & Policies)

Goal (simple): providers expose the right resources and run safely. **Requirements:**

- **RC/FA**: Tier-2 OK; $\geq$ 40GB VRAM if GPU is used; disk read $\geq$ 500MB/s recommended.
- **LA**: Tier-1; $\geq$ 80GB VRAM; `nvlink_required=true`; optional `gpus_per_host`$\geq$ 4. Fixed CUDA driver stack.

- **Sandbox**: non-root, read-only FS, egress allowlist (Relay + buyer buckets only).
- **Telemetry (for receipts/ops)**: `relations_unique`, `matrix_nnz`, `iters_done`, `checkpoint_at`, plus `gpu_time_hours_raw`.

C10 — QC & Disputes (Bit-Exact, Integer Job)

Goal (simple): acceptance/slashing rules that fit an integer, deterministic workload. **QC policy (v1 for NFS):**

- **Duplication**: 5% packages, but for LA duplication must use same tier and same `arch`.
- **Canaries**: $\sim 1\%$ small Ns for RC/FA/LA flows with known outputs; exact compare.
- **Spot re-exec**: 0.5% items for RC and FA; for LA, re-run K iterations from a checkpoint in neutral pool.

Acceptance (per package): canaries pass $\wedge$ dup matches (bit-exact) $\wedge$ spot matches $\Rightarrow$ accepted. **Dispute:** if dup mismatch, re-exec 10% of relations (RC/FA) or a block of LA iterations; final SQ verifies $N \bmod p = 0$ and, for RSA demo, $p \cdot q = N$.

C11 — Submission Flow & Buyer Guidance (CADO-Specific)

Goal (simple): make sending the CADO job to the platform low-friction and correct. **Flow:**

1. Run `ngh-pack cado` to produce the package index (RC*, FA*, LA, SQ).
2. In the portal, select region $\rightarrow$ deposit vouchers (Tier-2 for RC/FA, Tier-1 for LA/SQ) $\rightarrow$ submit `package_index.jsonl`.
3. The feasibility view shows: RC/FA parallelism and the single LA resource requirement (NVLink host).

Guidance:

- Default *bit-exact*; pin `arch` for LA to avoid cross-arch drift.
- Size RC shards so first counts flow in ≤ 120s; keep shard NGH between $5-15$.
- Choose providers with $\geq$ 80GB VRAM & NVLink for LA; otherwise expect misses or reschedules.

C12 — Demo Plan, Targets & Acceptance Tests

Goal (simple): prove end-to-end factorization on-platform with receipts. **Targets (choose one, sized for a 1-hour window with headroom):**

- **Primary demo:** $N = 320$–384-bit semiprime (toy-but-real); RC on Tier-2 in parallel; LA on one Tier-1 multi-GPU host.
- **Stretch:** 512-bit with longer LA; book a longer window or multiple hours with resume at checkpoints.

KPIs: time-to-first-relation ≤ 120s (RC), first LA checkpoint ≤ 120s, total accepted NGH within voucher deposit, final p, q verified. **Acceptance tests:**

1. **Bit-exact RC:** duplicate an RC shard on two providers; `nfs_relations@1` Merkle roots identical.
2. **FA stitch:** independent FA blocks merge to the same `nfs_matrix@1` root across two runs.
3. **LA determinism:** restart LA from checkpoint k on the same arch; dependency vector root identical.
4. **Final proof:** SQ emits `nfs_factors@1`; verifier checks $p \cdot q = N$ (for RSA demo) and records success in receipt bundle.

What this gives us. With C1–C12 in place, we (i) parallelize aggressively where safe (RC), (ii) keep global-dependence phases inside a single provider (LA/SQ), (iii) maintain integer, bit-exact comparisons for QC and disputes, and (iv) pay fairly via NFS-aware benchmarks. The result is a defensible, auditable showcase of decentralized compute delivering a nontrivial, mathematics-heavy workload.